

T-DID: Runtime Security Information and Stateful Threat Adjudication for DID/VC Systems

IEEE Publication Technology, *Staff*, *IEEE*,

Abstract—Decentralized Identifier (DID)와 Verifiable Credential (VC)의 보안 판정은 개별 서명 검증만으로 결정되지 않으며, 발급자, 지갑, 검증자와 지원 서비스에서 수행된 검사와 최종 처리 결과 및 관련 사건의 관계를 함께 확인해야 한다. 그러나 구현체마다 같은 보안 정보가 생성되는 위치와 기록 방식이 다르고, 기본 기록만으로는 특정 위협의 성공 조건을 완전하게 구성하기 어려울 수 있다. 본 연구는 DID/VC 1차 자료에서 선정한 21개 위협을 ATT&CK, AADAPT, CAPEC와 CWE에 대응시키고, 15개 고정 버전 구현체와 조합한 315개 위협-구현체 쌍에서 판정에 필요한 런타임 보안 정보, 기록 생산 주체와 사건 관계를 분석한다. ATT&CK와 다섯 비교 요소가 모두 일치한 위협은 21개 중 6개였으며, 315개 조합 중 151개는 적용 대상이 아니었고 나머지 164개에서는 기본 자료만으로 전체 성공 조건에 필요한 정보가 완성되지 않았다. 이를 바탕으로 구현체별 기록을 공통 보안 의미로 변환하고, 검사 결과, 최종 결정, 기록 출처, 사건 관계와 정보 부족을 함께 처리하는 T-DID를 제안한다. TDID-THR-015의 walt.id와 Credo TS 두 발급자에서 설계한 다섯 조건 중 세 조건을 대상으로 660개 워크플로를 평가한 결과, T-DID는 정보가 완전한 추가 발급 208개에서 세 평가 조건을 확인하고 정보가 부족한 추가 발급 240개를 별도로 구분했으며, 정상 실행 212개 중 210개에서는 조건이 성립하지 않았고 2개는 정보 부족으로 남았다. 시간적으로 분리한 220개 워크플로에서도 같은 결과 구분이 반복됐다. 또한 공통 및 생산자 로컬 정보 요소가 21개 위협의 판정 요구에서 추적 가능하게 도출됨을 확인했으며, 부하 실행에서는 낮은 부하 구간에서 제안 로깅과 판정을 적용한 상태로 워크플로를 처리할 수 있었고 높은 부하에서는 더 이른 포화과 메모리 증가가 관찰됐다. T-DID는 DID/VC 위협의 성공 조건을 실제 실행에서 생성되는 보안 정보와 연결하고, 조건 불성립과 정보 부족을 구분하여 판정 근거를 제공하는 런타임 보안 프레임워크를 제시한다.

Index Terms—Decentralized Identifiers, Verifiable Credentials, Runtime Security Monitoring, Security Events, Stateful Analysis.

I. INTRODUCTION

Decentralized Identifier (DID)와 Verifiable Credential (VC)은 중앙 신원 제공자에 대한 의존을 줄이면서 발급자, 소지자와 검증자 사이에서 검증 가능한 신원 정보를 교환하기 위한 기반을 제공한다. W3C DID와 VC 표준은 식별자, 검증 방법, 자격증명의 표현과 검증 절차를 정의하고, OpenID4VCI와 OpenID4VP는 자격증명의 발급과 제시 절차를 규정한다 [1]–[6]. 그러나 VC의 보안 결정은 서명 검증 하나로 끝나지 않는다. 발급 과정에서는 요청과 권한의 관계와 key proof의

유효성을 확인해야 하며, 제시 과정에서는 nonce와 세션의 일치, 자격증명 상태, 발급자 신뢰와 키 통제 여부 등을 함께 판단해야 한다. 검사 결과와 최종 발급 또는 수락 결정은 발급자, 지갑, 검증자, DID 해석기와 자격증명 상태 서비스 등으로 다른 구성요소에서 만들어질 수 있으므로, 특정 위협의 성공 여부를 판정하려면 관련 기록을 같은 실행이나 객체에 연결해야 한다.

ATT&CK, AADAPT, CAPEC와 CWE는 공격 행위, 공격 패턴과 소프트웨어 약점을 체계화하지만 [7]–[10], 분류 항목 자체가 특정 DID/VC 실행의 판정 조건을 제공하지는 않는다. 예를 들어 OpenID4VCI의 key-proof replay를 판단하려면 인증 자료가 재사용됐다는 일반적 설명을 넘어 동일한 proof의 재사용, proof 검증 결과, 정책상 허용된 사용 횟수와 실제 추가 발급 결과를 함께 확인해야 한다. 본 연구가 1차 자료에서 선정한 21개 위협을 공격 목적, 사전조건, 대상, 작동 방식과 성공 결과로 대조한 결과, ATT&CK와 다섯 요소가 모두 일치한 위협은 6개였고 나머지는 하나 이상의 실행 조건에서 차이가 있었다 [11]–[13]. 따라서 기존 분류체계는 DID/VC 위협을 외부 보안 지식과 연결하는 데 사용할 수 있지만, 실제 판정 조건은 DID/VC 위협의 성공 조건에서 별도로 도출해야 한다.

판정 조건을 정의하더라도 필요한 정보가 구현체에서 실제로 관측된다는 보장은 없다. CVE-2024-21669가 보고한 ACA-Py의 특정 LDP-VC presentation verification 경로에서는 challenge 검사 실패와 전체 verification 성공이 함께 나타날 수 있었다 [14], [15]. 최종 결과만 기록하면 중간 검사 실패를 확인하기 어렵고, proof replay나 presentation replay와 같이 여러 요청에 걸쳐 성립하는 위협은 이전 사건과 현재 사건을 연결할 상태도 필요하다. 구현체마다 같은 의미의 값이 서로 다른 함수나 구성요소에서 생성되고 일부 값은 기본 로그에 남지 않기 때문에, 로그를 중앙화하거나 공통 형식으로 바꾸는 것만으로 위협 판정에 필요한 정보가 완성되지는 않는다. OpenTelemetry, OCSF, ECS와 CloudEvents도 이벤트 표현과 정규화의 기반을 제공하지만, DID/VC 위협별 필수 정보, 해당 정보를 생성하는 구성요소와 판정에 필요한 사건 관계까지 정의하지는 않는다 [16]–[19].

본 연구는 DID/VC 위협의 성공 조건에서 필요한 런타임 보안 정보를 도출하고, 구현체에서 해당 정보가 생성되는 위치와 기록 여부를 분석한 뒤, 공통 보안 이벤트와 상태 기반

판정을 이용해 위협을 판단하는 T-DID를 제안한다. 공통 보안 이벤트는 구현체별 기록을 동일한 보안 의미로 변환하면서 검사 결과, 최종 결정, 기록 생산 주체, 사건 연결 정보와 부족한 값을 함께 표현한다. 상태 기반 판정은 현재 사건과 이전 사건을 연결해 위협별 성공 조건을 확인하며, 필요한 정보가 부족하면 이를 정상으로 처리하지 않고 기록 부족으로 구분한다. Fig. 1는 기존 위협 지식에서 실행 판정에 필요한 조건과 정보를 도출하고, 구현체별 기록을 T-DID의 판정 결과로 연결하는 전체 흐름을 보여준다.

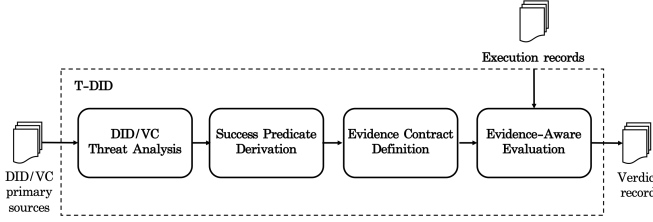


Fig. 1. Problem-solution overview of T-DID.

분석 대상은 발급자, 지갑, 검증자, 에이전트, DID 해석기와 자격증명 라이브러리를 포함한 15개 고정 버전 구현체다. 21개 위협과 15개 구현체의 315개 조합에서 위협 판정에 필요한 런타임 보안 정보, 기록 생산 주체와 사건 연결 관계를 분석하고, 확보 가능한 정보를 기준으로 통제된 실행 평가를 설계할 수 있는 범위를 구분한다. 이 분석은 취약점 수, 제품별 보안 수준 또는 탐지 정확도를 산출하기 위한 것이 아니다. 동적 평가는 TDID-THR-015의 walt.id와 Credo TS 두 발급자에 한정하며, 설계한 다섯 성공 조건 가운데 허용 사용 횟수 초과, 별도 자격증명 발급과 최종 발급 결정에 해당하는 세 조건을 대상으로 기본 기록의 정보 충분성, 기록 완전성에 따른 판정과 처리 비용을 평가한다.

본 연구는 다음 연구 질문에 답한다.

- RQ1.** DID/VC 위협은 ATT&CK, AADAPT, CAPEC와 CWE에 어느 범위까지 대응하며, 기존 분류체계만으로 설명하기 어려운 실행 조건은 무엇인가?
- RQ2.** DID/VC 위협의 성공을 판정하기 위해 어떤 런타임 보안 정보, 기록 생산 주체와 사건 간 연결 정보가 필요하며, 15개 고정 버전 구현체에서 해당 정보는 어디에서 얻을 수 있는가?
- RQ3.** TDID-THR-015의 두 발급자 사례에서 기본 기록은 평가 대상 세 조건에 충분한가, 그리고 추가 런타임 보안 정보와 상태 기반 판정을 적용할 때 평가 조건이 확인된 실행과 기록이 부족한 실행을 구분할 수 있는가?
- RQ4.** 두 발급자에서 평가에 필요한 런타임 보안 정보는 어느 처리 경로에서 얻을 수 있으며, 로그 생성부터 T-DID 판정까지의 처리 비용은 어느 정도인가?

본 논문의 주요 기여는 세 가지다.

- 1) 1차 자료에서 선정한 21개 DID/VC 위협을 ATT&CK, AADAPT, CAPEC와 CWE에 대응시키고, 실제 실행 판정을 위해 추가로 확인해야 하는 DID/VC 조건을 분석한다.
- 2) 21개 위협과 15개 고정 버전 구현체의 315개 조합을 분석하여 위협 판정에 필요한 런타임 보안 정보, 기록 생산 주체와 사건 연결 관계를 도출하고, 통제된 실행 평가를 설계할 수 있는 범위를 구분한다.
- 3) 구현체별 기록을 정규화하는 공통 보안 이벤트와 상태 기반 판정 구조를 갖는 T-DID를 제안하고, TDID-THR-015의 두 발급자와 세 평가 조건에서 기록 완전성에 따른 판정과 처리 비용을 평가한다.

제2장은 DID/VC 보안 결정, 런타임 모니터링, 보안 이벤트 모델과 공격 분류체계에 관한 관련 연구를 정리한다. 제3장은 위협과 구현체의 런타임 보안 정보를 분석하고, 제4장은 T-DID의 공통 보안 이벤트와 상태 기반 판정 구조를 설명한다. 제5장은 프로토타입과 구현체 연동을 제시하며, 제6장은 315개 조합의 실행 평가 가능성과 TDID-THR-015 사례를 평가한다. 제7장과 제8장은 적용 범위와 한계 및 결론을 다룬다.

II. BACKGROUND AND RELATED WORK

A. DID/VC Security Decisions and Runtime Observability

DID/VC의 발급과 제시 과정에서는 여러 구성요소가 서로 다른 보안 검사를 수행한다. OpenID4VCI 발급자는 자격증명 요청과 key proof를 검증하고, OpenID4VP 검증자는 presentation response가 요청된 nonce와 대상에 부합하는지 확인한다 [5], [6]. VC 검증에서는 발급자 신뢰, 자격증명 상태와 자격증명에 연결된 키의 통제 여부도 고려되며, DID 해석은 검증에 필요한 verification method와 해석 메타데이터를 제공한다 [1]–[4].

표준에 검사 절차가 정의되어 있다는 사실과 특정 실행에서 해당 검사가 실제로 수행되어 최종 결정에 반영됐다는 사실은 구분해야 한다. 실행 단계의 판정에는 검사에 사용된 값, 검사 수행 여부와 결과, 최종 발급 또는 수락 결정이 필요하다. nonce, proof 또는 authorization code의 재사용처럼 둘 이상의 요청에 걸쳐 성립하는 조건은 이전 실행과 현재 실행을 연결할 정보도 요구한다 [5], [6], [20]. 원문 credential이나 proof 전체를 저장하지 않고도 동일성 또는 사건 관계를 확인하기 위한 제한된 참조값을 이용해 필요한 상태를 유지할 수 있다. 따라서 DID/VC의 런타임 보안 판정에서는 프로토콜 메시지를 수집하는 것만큼 검사 결과, 최종 결정과 사건 관계를 어떤 실행 정보로 확인할 수 있는지가 중요하다.

B. Stateful Runtime Monitoring

런타임 모니터링은 실행 중 관측한 사건을 미리 정의된 명세 또는 보안 속성과 비교하여 실행 상태를 판단한다. 명세 기반 런타임 모니터링에 관한 연구는 사건 관측, 상태 관리,

명세 표현과 판정 방법을 폭넓게 다뤄 왔으며, 한 시점의 사건만으로 판단하기 어려운 속성에는 사건의 순서, 반복 횟수와 누적 상태가 필요함을 보여준다 [21]. 필요한 관측값이 부족할 때 명확한 참 또는 거짓으로 결론 내리지 않는 판정의 의미도 런타임 검증 분야에서 다뤄져 왔다.

PHOENIX는 모바일 기기의 셀룰러 제어 트래픽에서 사건 순서를 관측하고 상태 기계 기반 시그니처와 비교하여 비정상 동작을 탐지한다 [22]. AP2를 대상으로 한 연구는 실행 요청이 위임된 문맥에 부합하는지 확인하고 시간 제한 nonce의 소비 상태를 유지하여 재사용을 판정한다 [23]. 적용 환경과 보안 목표는 다르지만 두 연구 모두 현재 사건만으로 확인할 수 없는 속성을 과거 상태와의 관계를 통해 판정한다.

보안 요구사항을 실행 가능한 조건으로 변환하는 연구도 수행되어 왔다. Farrell 등은 위협 모델에서 정의한 보안 속성을 형식화하여 정적 검증과 런타임 모니터로 연결했고, Zdun 등은 보안 권고의 준수 여부를 모델 기반 매트릭 규칙으로 표현하여 전문가 판단과 비교했다 [24], [25]. DID 분야에서는 Huh 등이 DID 해석 과정에서 발생할 수 있는 개인정보 노출과 DID document cache 부채널을 분석하고, 요청된 DID와 해석 결과의 기밀성을 보호하는 Oblivira를 제안했다 [26]. 선행연구는 상태를 이용한 실행 판정, 보안 요구사항의 규칙화와 DID 해석 단계의 보호 방법을 각각 제시한다.

C. Security Event and Provenance Models

여러 구성요소에서 생성된 기록을 함께 분석하려면 필드의 의미와 사건 문맥을 정규화할 필요가 있다. OpenTelemetry Logs Data Model은 관측 자료를 공통 로그 레코드로 표현하는 구조를 제공하고, OCSF는 보안 사건을 공통 category와 class로 정규화한다 [16], [17]. Elastic Common Schema는 로그, metric과 trace에서 사용하는 공통 필드의 명명 규칙을 제공하며, CloudEvents는 서로 다른 시스템 사이에서 사건 문맥과 데이터를 교환하기 위한 형식을 정의한다 [18], [19].

사건 사이의 관계와 기록의 출처를 표현하기 위한 모델도 존재한다. STIX 2.1은 사이버 위협 객체와 관계 및 관측 자료를 구조화하고, PROV-DM은 entity, activity와 agent 사이의 출처 관계를 정의한다 [27], [28]. W3C Trace Context는 분산된 요청을 동일한 trace로 연결하기 위한 trace-id와 parent 관계를 표준화한다 [29].

범용 모델은 이질적인 기록의 형식, 출처와 요청 관계를 표현하는 기반을 제공하지만, 특정 DID/VC 위협의 성공을 판정하기 위해 어떤 값이 반드시 필요한지 또는 어느 구성요소가 해당 값을 생성해야 하는지까지 정하지 않는다. 예를 들어 proof replay에서는 일반적인 timestamp와 trace identifier만으로 충분하지 않으며, proof의 동일성, 검증 결과, 허용 사용 횟수와 실제 발급 결과를 함께 확인해야 한다. 필요한 값이 존재하지 않는 경우를 조건 불일치와 어떻게 구분할지도 위협별 판정 조건에 따라 별도로 정의해야 한다.

D. Threat Models and Attack Taxonomies

W3C Decentralized Credentials Threat Model은 credential lifecycle과 생태계의 trust boundary를 중심으로 보안, 개인 정보 보호와 사회기술적 위협을 정리한다 [11]. DID/VC의 공격 표면과 신뢰 관계를 파악하는 직접적인 근거를 제공하지만, 개별 구현에서 위협의 성공 여부를 판정하기 위한 실행 정보나 기록 위치를 정의하지는 않는다.

범용 보안 분류체계는 서로 다른 목적과 추상화 수준을 갖는다. ATT&CK는 공격자의 tactic과 technique을 중심으로 공격 행위를 구조화하고, AADAPT는 digital asset와 payment technology에 관련된 적대 행위를 다룬다 [7], [8]. CAPEC는 약점을 악용하는 공격 패턴을 기술하고, CWE는 구현 단계에서 발생하는 software 및 hardware weakness를 분류한다 [9], [10]. 같은 DID/VC 위협도 공격 행위, 공격 패턴 또는 구현 약점 가운데 어떤 관점에서 비교하는지에 따라 대응 항목과 대응 수준이 달라질 수 있다.

분류 항목과 실행 판정 조건은 같은 정보를 요구하지 않는다. 공격 목적이나 작동 방식이 유사하더라도 DID/VC 프로토콜에서는 특정 credential, proof, nonce, session 또는 DID 해석 결과와의 관계를 추가로 확인해야 할 수 있다. 반대로 소프트웨어 약점이 정확히 대응하더라도 약점의 존재만으로 해당 실행에서 공격이 성공했다고 판단할 수는 없다. 따라서 위협의 사전조건, 대상, 작동 방식과 성공 결과는 DID/VC 표준과 공개 취약점 등 1차 자료에서 먼저 확인하고, 기존 분류체계와의 대응은 선정된 위협을 외부 보안 지식과 연결하는 별도의 단계로 다룰 필요가 있다.

관련 연구를 종합하면 세 가지 기반이 확인된다. 런타임 모니터링 연구는 사건과 상태를 실행 가능한 보안 판정에 연결하는 방법을 제공하고, 보안 이벤트 및 출처 모델은 여러 구성요소의 기록을 표현하고 연결하는 구조를 제공하며, 위협 모델과 공격 분류체계는 공격 표면과 기존 보안 지식을 체계화한다. 그러나 DID/VC 위협의 성공 조건에서 필요한 런타임 보안 정보를 도출하고, 해당 정보를 생성하는 구성요소와 구현체별 위치를 확인한 뒤, 여러 사건의 상태와 최종 보안 결과를 결합하여 판정하는 과정은 하나의 흐름으로 다루지지 않았다. T-DID는 이 공백을 대상으로 위협별 성공 조건과 런타임 보안 정보, 공통 보안 이벤트와 상태 기반 판정을 연결하며, 판정에 필요한 정보가 부족한 경우를 성공 조건이 성립하지 않은 경우와 구분한다.

III. THREAT ANALYSIS AND RUNTIME SECURITY INFORMATION

본 장은 DID/VC 위협 기준선을 구성하고 기존 보안 분류체계와의 대응 관계를 분석한 뒤, 각 위협의 성공 여부를 실행 중에 판정하기 위해 필요한 런타임 보안 정보와 기록 생산 주체를 도출한다. 이어서 15개 고정 버전 구현체의 처리 경로에서 해당 정보가 어디에서 생성되고 기본 기록 또는 실행 상태로 확보 가능한지 분석한다.

A. Threat Baseline Construction

위협 기준선의 범위는 DID 문서와 DID 해석, 자격증명 상태, 발급 및 제시 과정으로 정했다 [1]–[6], [11]. 후보 위협은 1차 자료에서 사전조건, 작동 방식과 성공 결과를 확인할 수 있고, 정상 동작과 공격 성공을 구분하는 데 필요한 정보와 해당 정보를 생성하는 주체를 식별할 수 있을 때 선정했다. DID/VC 절차와 직접 연결되지 않는 일반 플랫폼 문제, 검사 의무의 누락만으로 정의되는 항목, 실행 기록만으로 공격 성공을 구분할 수 없는 항목은 제외했다.

초기 후보 31개 가운데 21개를 선정하고 10개를 제외했으며 보류한 후보는 없었다 [12]. 선정된 위협에는 TDID-THR-001부터 TDID-THR-021까지 고정 식별자를 부여했으며, 식별자는 검토 순서를 나타낼 뿐 심각도나 구현체별 결과를 의미하지 않는다. 후보별 선정·제외 근거는 보충 Table S0에, 21개 위협의 정의와 성공 조건 및 1차 근거는 보충 Table S1에 제시한다.

Algorithm 1 Threat and runtime-information analysis.

Require: primary materials P , taxonomies X , fixed implementations I
Ensure: threats H , taxonomy map M , runtime-information map R

```

1: for each candidate  $c$  derived from  $P$  do
2:    $q \leftarrow \text{EXTRACT}(c, \text{goal}, \text{precondition}, \text{target}, \text{mechanism}, \text{success})$ 
3:   if  $q$  is DID/VC-specific and success is distinguishable at runtime then
4:      $H \leftarrow H \cup \{c\}$ 
5:   end if
6: end for
7: for each threat  $h \in H$  do
8:    $M[h] \leftarrow \text{RECONCILE}(h, X, \text{five elements})$ 
9:    $(F, A, J) \leftarrow \text{DERIVERUNTIMEINFORMATION}(h)$ 
10:  for each implementation  $i \in I$  do
11:     $R[h, i] \leftarrow \text{INSPECTFIXEDPATH}(i, F, A, J)$ 
12:    record unavailable items without imputation
13:  end for
14: end for
15: return  $H, M, R$ 
```

Algorithm 1은 위협 선정부터 분류체계 대조와 구현체별 런타임 보안 정보 분석까지의 절차를 요약한다. 먼저 각 위협의 목적, 사전조건, 대상, 작동 방식과 성공 결과를 1차 자료에서 정의한 뒤 기존 분류체계와 대조한다. 이후 성공 조건을 판정하는 데 필요한 정보를 기록 생산 주체와 사건 관계에 연결하고, 고정 구현체의 처리 경로에서 각 정보의 생성 위치와 기록 여부를 확인한다. 필요한 정보가 확인되지 않으면 값을 추정하지 않고 확보되지 않은 정보로 기록한다.

B. Taxonomy Reconciliation

선정한 21개 위협은 공격 목적, 사전조건, 대상, 작동 방식과 성공 결과의 다섯 요소를 기준으로 ATT&CK, AADAPT, CAPEC와 CWE의 관련 항목과 대조했다 [7]–[10], [13]. 다섯 요소가 모두 일치하면 정확 대응으로 분류하고, 목적·작동 방식·성공 결과는 일치하지만 사전조건이나 대상 범위가 다르면 부분 대응으로 분류했다. 목적이나 결과 수준의 유사성은 있으나 핵심 사전조건 또는 작동 방식이 다르면 유사 대응으로, 작동 방식과 성공 결과를 직접 연결할 항목을 확인하지 못하면 대응 없음으로 분류했다.

기존의 technique 중심 ATT&CK 검토는 DID 환경에서 관련될 수 있는 항목을 폭넓게 확인하기 위한 진단 자료로만 사용했으며, 해당 행 수나 기존 라벨을 최종 위협 대응 결과로 합산하지 않았다. 최종 대응은 선정된 21개 위협 각각을 다섯 요소로 다시 대조한 위협 단위 결과를 사용한다 [13].

Table I은 21개 위협의 ATT&CK, AADAPT, CAPEC와 CWE 대응 결과를 제시한다. ATT&CK에서는 정확 대응 6개, 부분 대응 5개, 유사 대응 7개, 대응 없음 3개로 나타났다. 대응 등급은 분류체계의 완성도나 우열을 의미하지 않으며, 각 DID/VC 위협과 해당 항목이 다섯 비교 요소에서 어느 범위까지 일치하는지를 나타낸다.

대응 수준의 차이는 실행 단계에서 추가로 확인해야 할 정보를 보여준다. TDID-THR-003의 post-verification DID substitution은 ATT&CK T1565.001과 다섯 요소가 모두 일치하지만, 실제 판정에서는 검증한 DID 문서와 이후 소비한 문서가 같은 객체인지 연결해야 한다. TDID-THR-002의 post-compromise key acceptance는 T1078과 작동 방식과 성공 결과가 일치하지만 권위 있는 침해 신호와 유효 시점이 추가로 필요하다. TDID-THR-001의 pairwise DID linkability는 T1589와 목적 수준에서는 유사하지만 명시적인 unlinkability 정책과 교차 문맥의 동일성 비교를 요구한다. TDID-THR-005의 DID URL parameter injection은 ATT&CK에서 같은 작동 방식과 성공 결과를 갖는 항목을 확인하지 못했다.

같은 위협도 분류체계에 따라 대응 수준이 달랐다. TDID-THR-015는 ATT&CK T1212와 AADAPT ADT3012.006에는 부분 대응하고 CAPEC-60에는 유사 대응하지만 CWE-294와는 정확 대응한다. CWE-294는 이전에 전송된 유효한 인증 자료가 freshness 또는 일회성 사용 검사의 실패로 다시 수락되는 작동 방식을 다루는 반면, AADAPT와 CAPEC 항목은 각각 블록체인 거래 서명과 세션 식별자를 전제로 한다. 전체 다섯 요소의 ATT&CK 비교 근거는 보충 Table S2에, AADAPT·CAPEC·CWE의 항목별 근거는 보충 Table S3에 제시한다.

RQ1의 분석 결과, 기존 분류체계는 DID/VC 위협을 알려진 공격 행위와 약점에 연결하는 데 사용할 수 있지만 분류 항목만으로 실행 판정 조건을 정할 수는 없었다. 정확 대응에서도 DID/VC 실행에서 비교할 객체와 사건 관계를 구체화해야 했고, 부분·유사 대응에서는 사전조건, 대상 또는 작동 방식의 차이를 추가 런타임 보안 정보로 표현해야 했다.

C. Runtime Information and Producer Analysis

구현체 분석은 발급자, 지갑, 검증자, 에이전트, DID 해석기와 자격증명 라이브러리의 서로 다른 역할과 처리 경로를 포함하도록 공개 구현체 15개를 목적 표집하고 각 저장소의 revision을 고정하여 수행했다 [30]. Table II은 분석한 구현체의 역할, 고정 revision과 본 논문에서 사용한 범위를 정리한다.

TABLE I
CROSS-TAXONOMY MAPPINGS OF THE 21 DID/VC THREATS.

ID	Threat scope	ATT&CK	AADAPT	CAPEC	CWE
THR-001	Pairwise DID linkability	T1589 (A)	ADT3025 (P)	–	CWE-359 (P)
THR-002	Post-compromise key acceptance	T1078 (P)	ADT1552.004 (A)	CAPEC-560 (P)	CWE-672 (P)
THR-003	Post-verification DID substitution	T1565.001 (E)	–	CAPEC-29 (E)	CWE-367 (E)
THR-004	Equivalent-URL cache exhaustion	T1499.003 (E)	ADT3009 (A)	CAPEC-125 (P)	CWE-770 (E)
THR-005	DID URL parameter injection	–	–	CAPEC-137 (E)	CWE-116 (E)
THR-006	Tampered external resource	T1565 (E)	ADT3012.004 (P)	CAPEC-148 (E)	CWE-345 (E)
THR-007	Dereferencing cycle	T1499.004 (P)	ADT3009 (A)	CAPEC-130 (E)	CWE-674 (E)
THR-008	Duplicate-parameter differential	–	–	CAPEC-460 (P)	CWE-436 (E)
THR-009	Cache-bypass denial of service	T1499.003 (E)	ADT3009 (A)	CAPEC-125 (E)	CWE-400 (E)
THR-010	relativeRef path traversal	–	–	CAPEC-139 (E)	CWE-706 (E)
THR-011	Out-of-session status tracking	T1119 (A)	ADT3025 (A)	CAPEC-212 (P)	CWE-359 (E)
THR-012	Issuer status correlation	T1589 (A)	ADT3025 (A)	CAPEC-212 (P)	CWE-359 (E)
THR-013	Active content execution	T1203 (E)	ADT3010 (A)	CAPEC-63 (E)	CWE-79 (E)
THR-014	Stable credential linkability	T1589 (A)	ADT3025 (A)	–	CWE-359 (P)
THR-015	Key-proof replay issuance	T1212 (P)	ADT3012.006 (P)	CAPEC-60 (A)	CWE-294 (E)
THR-016	Split-wallet deputy impersonation	T1550 (P)	ADT3018 (A)	CAPEC-151 (E)	CWE-287 (E)
THR-017	Transaction-code phishing	T1598 (A)	–	CAPEC-98 (E)	CWE-346 (E)
THR-018	Repeated DCQL probing	T1589 (P)	ADT3027 (A)	CAPEC-462 (P)	CWE-203 (E)
THR-019	direct_post session fixation	T1557 (A)	–	CAPEC-61 (E)	CWE-384 (E)
THR-020	Cross-issuer claim collision	T1078 (A)	–	CAPEC-151 (P)	CWE-287 (P)
THR-021	Cross-transaction presentation replay	T1212 (E)	ADT3012.006 (P)	CAPEC-60 (P)	CWE-294 (E)

E: exact; P: partial; A: analogous; –: no corresponding entry identified.

TABLE II
FIXED IMPLEMENTATIONS USED IN THE ANALYSIS.

Implementation	Role(s)	Fixed revision	Scope
walt.id identity [31]	Issuer, verifier, wallet	f5b5995	Static analysis + THR-015 evaluation
ACA-Py [32]	Issuer, holder, verifier, mediator, cloud agent	b5bf838	Static analysis
Universal Resolver [33]	Resolver	844752e	Static analysis
EUDI Verifier Endpoint [34]	Verifier, relying party	v0.11.0	Static analysis
EUDI PID Issuer [35]	Issuer	c14af67	Static analysis
Credo TS [36]	Issuer, holder, verifier, agent, resolver	3c32928	Static analysis + THR-015 evaluation
EUDI Android Wallet Core [37]	Wallet, holder	c67b0eb	Static analysis
Bifold Wallet [38]	Wallet, holder, DIDComm edge	41c2c54	Static analysis
Multipaz Wallet [39]	Wallet, holder, backend	a0d63e6	Static analysis
Hyperledger Identus Cloud Agent [40]	Issuer, holder, verifier, cloud agent	79ca64e	Static analysis
Veramo [41]	Issuer, holder, verifier, agent	3008904	Static analysis
DIF JavaScript DID Resolver [42]	Resolver library	a36ebdb	Static analysis
SpruceID SSI [43]	Credential, resolver library	8963036	Static analysis
SpruceID OpenID4VP [44]	Verifier, wallet library	e5f29b8	Static analysis
Digital Bazaar vc-js [45]	Credential, presentation library	7ddb985	Static analysis

Bifold Wallet은 Credo를 사용하고 SpruceID OpenID4VP는 SpruceID SSI와 코드를 공유하므로, 해당 관계를 서로 독립적인 구현 관측으로 해석하지 않았다 [30]. 분석은 각 위협의 성공 조건을 실행 중 확인해야 하는 정보로 나눈 뒤, 해당 정보를 생성하는 구성요소, 같은 요청이나 객체를 연결하는 관계, 여러 사건에 걸쳐 유지해야 하는 상태와 최종 보안 결과를 정리하는 방식으로 수행했다. 이후 고정 소스의 처리 경로에서 각 값이 생성되는 함수, 반환값과 내부 상태 및 기본 기록 여부를 확인했다.

코드에서 일부 정보를 확인하더라도 다른 필수 정보가 외부 구성요소나 실행 상태에서만 생성된다면 판정에 필요한 정보가 완성된 것으로 보지 않았다. 또한 특정 코드 위치에서 검사가 확인되지 않거나 후보 경로가 존재한다는 사실만으

로 공격 성공, 취약점, 탐지 성능 또는 구현체의 보안 수준을 판단하지 않았다. 고정 소스의 처리 경로와 통제된 실행 평가 가능성은 별개의 분석 결과다.

21개 위협 가운데 11개는 성공 조건에 필요한 정보를 한 운영 주체의 범위에서 생성할 수 있었고, 10개는 둘 이상의 구성요소 또는 기관에서 생성된 정보를 연결해야 했다 [12], [46]. TDID-THR-015에서는 발급자가 proof 검증과 자격증명 발급 결과를 생성하지만, 같은 proof의 반복 사용 여부를 판정하려면 이전 발급과 현재 발급을 연결하여 사용 이력과 서로 다른 발급 결과를 비교해야 한다. TDID-THR-019에서는 요청을 시작한 검증자 세션, 지갑이 처리한 응답, 응답을 받은 세션과 최종 접근 결정을 여러 구성요소의 기록에서 연결해야 한다.

같은 의미의 정보도 구현체마다 생성 위치가 달랐다. TDID-THR-015의 경우 `walt.id`의 고정 버전에서는 `DefaultCredentialProofVerifier`가 `key proof`를 검증하고, 이후 `OpenId4VciProtocolService`가 자격증명 발급과 세션 종료 상태를 처리한다 [31]. Credo TS의 고정 버전에서는 `OpenId4VcIssuerService` 내부의 서로 다른 처리 구간에서 `proof` 검증, 발급 결과와 세션 상태가 생성된다 [36]. 따라서 구현체별 필드명이나 함수 위치를 직접 판정 규칙에 사용하는 대신, 값의 보안 의미와 기록 생산 주체 및 사건 관계를 기준으로 연결해야 한다.

21개 위협과 15개 구현체를 결합한 315개 조합에서는 위협의 적용 여부, 필요한 정보가 생성되는 고정 처리 경로와 외부 또는 위임된 기록 생산 주체를 확인했다. 전체 조합의 세부 분석은 보충 Table S4b에, 확인한 파일과 코드 위치는 보충 Table S4c에 제시한다. 해당 분석은 런타임 보안 정보의 생산 위치와 확보 경계를 설명하기 위한 것이며, 315개 조합에서 취약점이나 공격 성공을 측정한 결과가 아니다. 통제된 실행 평가를 설계할 수 있는지에 대한 별도 판정과 사례 선정 결과는 제6장에서 다룬다.

RQ2의 분석 결과, DID/VC 위협의 성공을 판정하려면 개별 검사 결과뿐 아니라 최종 발급·수락·접근 결과, 값을 생성한 구성요소, 같은 요청이나 객체를 식별하는 사건 관계와 이전 사건의 상태가 필요했다. 또한 같은 의미의 정보도 구현체마다 생성 위치와 기본 기록 여부가 달랐다. 제4장은 해당 분석 결과를 바탕으로 구현체별 기록을 공통 보안 이벤트로 변환하고 사건 간 상태를 이용해 위협의 성공 조건을 판정하는 T-DID의 구조를 설명한다.

IV. T-DID DESIGN

T-DID는 제3장에서 도출한 위협별 런타임 보안 정보를 구현체에 독립적인 공통 보안 이벤트로 변환하고, 관련 사건의 상태를 유지하여 위협의 성공 조건을 판정한다. 분류체계 식별자는 위협을 기존 보안 지식과 연결하는 데만 사용하며 판정 규칙에는 포함하지 않는다. 각 규칙은 DID/VC 1차 자료에서 정의한 성공 조건을 검사 결과, 기록 생산 주체, 사건 관계와 최종 보안 결과에 대응시켜 구성한다.

A. System Overview

Fig. 2은 제1장의 개념적 흐름을 T-DID의 실제 구성요소와 데이터 흐름으로 구체화한다. 상단의 규칙 도출 과정에서는 위협별 성공 조건을 판정에 필요한 런타임 보안 정보, 해당 정보를 생성하는 주체와 사건 간 관계로 분해한 뒤 판정 규칙을 정의한다. 이 과정은 실행 전에 수행되며, ATT&CK, AADAPT, CAPEC와 CWE의 대응 결과는 규칙 입력으로 사용하지 않는다.

하단의 실행 과정에서는 발급자, 지갑, 검증자, DID 해석기와 상태 서비스에서 생성되는 로그, 검사 결과와 최종 처리

결과를 어댑터가 수집한다. 어댑터는 구현체마다 다른 기록을 공통 보안 이벤트의 필드에 대응시키며, 원본 기록에 없는 값을 추정하거나 기본값으로 생성하지 않는다. 변환된 이벤트는 상태 저장소에서 같은 요청, 세션 또는 객체의 이전 사건과 연결되고, 판정 엔진은 현재 이벤트와 저장된 상태를 위협별 규칙과 비교한다. 결과에는 최종 판정뿐 아니라 조건별 결과와 판정에 사용한 기록 또는 부족한 정보가 포함된다.

상태 저장소는 모든 이벤트를 하나의 전역 이력으로 결합하지 않는다. 규칙에서 요구하는 범위에 따라 요청, 객체 또는 제한된 참조값을 기준으로 필요한 사건만 연결한다. 단일 구성요소에서 필요한 정보를 생성할 수 있는 위협은 해당 운영 주체의 로컬 상태로 처리하고, 여러 구성요소의 정보가 필요한 경우에는 각 값을 생성한 주체와 허용된 사건 관계를 확인한 뒤 결합한다. 연결 범위를 확인할 수 없는 값은 다른 사건의 값으로 보완하지 않는다.

T-DID는 기존 발급·제시·검증 경로 밖에서 동작하며 구현체의 보안 결정을 변경하지 않는다. 식별 가능한 기록 생산 주체와 보호된 수집 경로를 신뢰 범위로 두고, 형식과 생산 주체 및 기록 무결성을 확인한 입력만 상태와 판정에 사용한다. 생산자가 실제 처리와 다른 내용을 기록하거나 사건 자체를 숨기는 경우는 이 범위에서 다루지 않는다. 로그 생성과 수집 경로에 대한 기본 가정은 기존 로그 관리와 감사 기록의 신뢰 모델을 따른다 [47]–[49].

B. Canonical Security Event

공통 보안 이벤트는 구현체별 필드명이나 코드 위치를 판정 규칙에서 직접 참조하지 않도록 실행 문맥, 검사 결과, 최종 결정, 사건 연결과 기록 상태를 공통 의미로 표현한다. 전체 필드 구조는 보충 Table S8에 제시하며, 본문에서는 판정에 직접 사용되는 필드 그룹만 설명한다.

실행 문맥에는 사건 식별자와 발생 시각, 프로토콜, 구성요소 역할, 구현체와 적용 정책을 기록한다. 해당 정보는 서로 다른 단계나 구성요소에서 생성된 기록을 구분하고, 규칙이 적용되는 정책과 구현 범위를 확인하는 데 사용된다. `checks`는 정책상 필요한 검사, 실제 수행된 검사와 검사별 결과를 구분하며, `final_decision`은 발급, 수락, 거부와 같은 최종 처리 결과를 별도로 기록한다. 따라서 개별 검사의 성공이나 실패가 전체 처리 결과와 동일한 값으로 축약되지 않는다.

사건 연결에는 목적에 따라 범위가 제한된 참조값을 사용한다. 예를 들어 `proof_scoped_ref`는 서로 다른 발급 사건에서 동일한 `proof`가 사용됐는지를 비교하기 위한 값이고, `event_id`는 같은 이벤트가 중복 전달됐는지를 식별하기 위한 값이다. 두 값은 목적이 다르므로 서로 대체하지 않는다. 원문 `proof`, `credential` 또는 접근 토큰은 공통 이벤트에 저장하지 않고, 동일성 비교가 필요한 경우에는 해당 운영 범위에서만 사용할 수 있는 참조값을 유지한다.

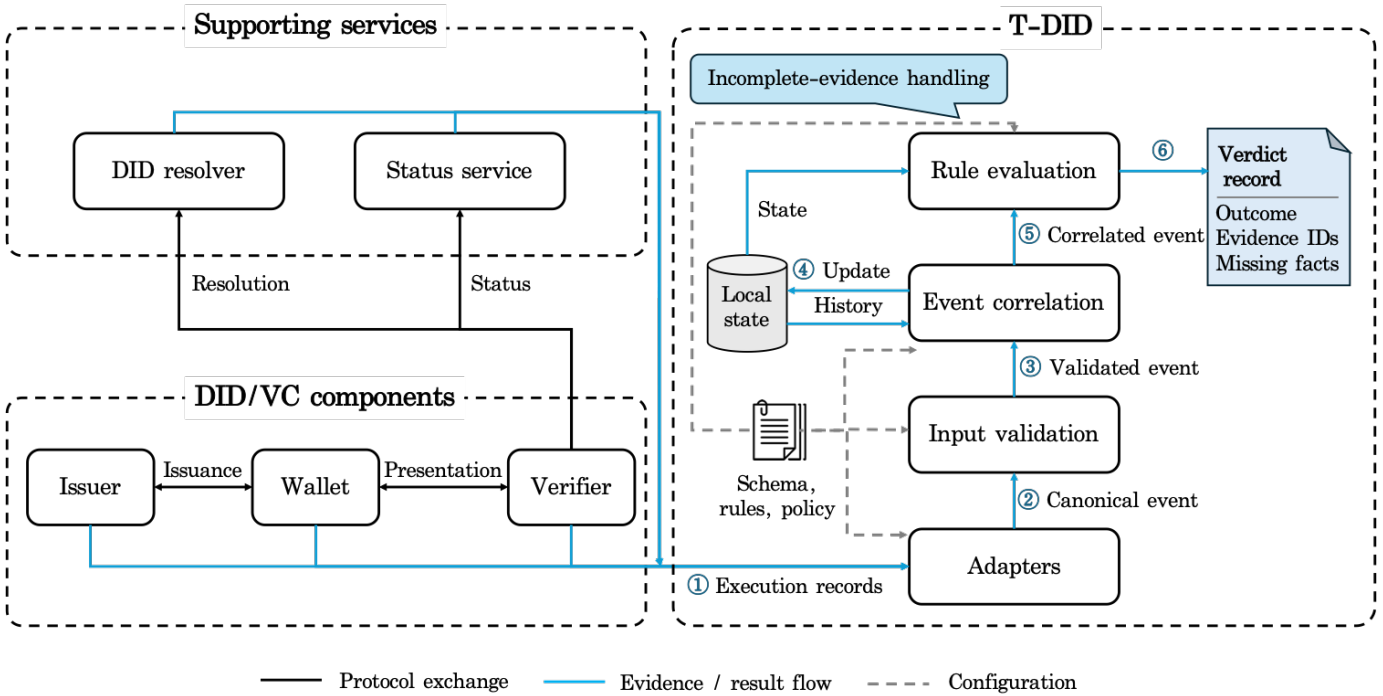


Fig. 2. System overview of T-DID.

`field_provenance`는 각 값이 기본 기록, 추가 계측, 로컬 계산, 외부 구성요소 또는 정책에서 얻어졌는지를 나타낸다. `evidence_integrity_result`는 수집된 기록에 대한 무결성 확인 결과를 전달하고, `missing_evidence`는 판정에 필요하지만 확보하지 못한 값과 해당 값을 생성해야 하는 구성요소를 기록한다. 출처를 확인할 수 없거나 필요한 값이 존재하지 않는 경우에는 판정 규칙에서 값을 임의로 보완하지 않는다.

공통 이벤트는 위험별 모든 상태를 직접 저장하지 않는다. 반복 횟수, 집합의 크기 또는 이전 사건과의 관계처럼 여러 실행에 걸쳐 계산해야 하는 값은 상태 저장소가 관리한다. 상태의 범위와 중복 처리 기준도 규칙에 따라 다르다. 예를 들어 `event_id`는 동일한 이벤트가 상태를 두 번 갱신하는 것을 막는 데 사용되지만, 동일 `proof`의 수락 횟수는 의미상 재시도와 중복 전달을 구분할 수 있는 발급자 로컬 식별 정보와 발급 결과를 기준으로 유지해야 한다. 전체 등록 검사와 상태 정의는 보충 Table S9와 S10에 제시한다.

C. Stateful Verdict Rules

각 판정 규칙은 제3장에서 정의한 위협의 성공 조건을 공통 보안 이벤트와 상태의 값에 대응시킨다. 현재 사건에서 확인할 수 있는 조건은 검사 결과와 최종 결정에서 읽고, 재사용 횟수, 사건 순서 또는 이전 처리 결과처럼 여러 실행에 걸친 조건은 상태 저장소에서 계산한다. 규칙은 보안 검사의 실패만 확인하는 것이 아니라, 해당 조건이 추가 발급, 잘못된 수락 또는 접근 승인과 같은 위협의 성공 결과로 이어졌는지

까지 확인한다. 21개 위협의 전체 성공 조건과 필요한 기록 생산 주체는 보충 Table S5에 제시한다.

Algorithm 2은 공통 이벤트를 이전 상태와 연결하고 판정 결과를 생성하는 절차를 보여준다.

Algorithm 2 Stateful threat evaluation in T-DID.

Require: canonical event e , threat rule r , policy π , state S

Ensure: verdict record v

$$1: (ok, c, m) \leftarrow \text{VALIDATEINPUT}(e, \pi)$$
2: if not *ok* then

```
3:   $o \leftarrow \text{input\_rejected}$ 
```

4: **else**5: $j \leftarrow \text{CORRELATE}(e, S, \pi)$

6: **if** $e.event_id$ has not updated S **then**

7: $S \leftarrow \text{UPDATESTATE}(\bar{S}, j, e, \pi)$ 8: **end if**

9: $(s, c, m) \leftarrow \text{EVALUATE}(r, e, S, \pi)$

10: **if** $s = \text{ERROR}$ **then**

```
11:       $o \leftarrow \text{engine\_error}$ 
```

12: **else if** all success conditions in r hold **then**

```
13:    $o \leftarrow \text{threat\_witness}$ 
```

14: **else if** $m \neq \emptyset$ **then**

```
15:    $o \leftarrow \text{incomplete\_evidence}$ 
```

16: **else if** a required check fails without the threat-success outcome **then**

```
17:       $o \leftarrow \text{obligation\_violation}$ 
```

```
18:  else
```

```
19:       $o \leftarrow \text{no\_witness}$ 
```

20: **end if**21: **end if**22: $v \leftarrow \text{RECORD}(o, c, m, \text{source references}, \pi)$ 23: **return** v

입력의 형식, 기록 생산 주체 또는 무결성을 확인하지 못하면 `input_rejected`를 반환하고 해당 입력으로 상태를 갱신하지 않는다. 유효한 이벤트는 같은 요청이나 객체의 이전 사건과 연결하며, 동일한 `event_id`가 다시 수집되더라도 상태는 한 번만 갱신한다. 규칙 엔진 자체가 평가를 완료하지

못한 경우는 `engine_error`로 구분한다.

위협의 성공 조건과 최종 보안 결과가 모두 확인되면 `threat_witness`를 기록한다. 판정에 필요한 정보가 존재하지만 성공 조건이 성립하지 않으면 `no_witness`, 성공 여부 또는 음성 조건을 판단하는 데 필요한 값이 부족하면 `incomplete_evidence`를 기록한다. 필수 검사가 실패하거나 수행되지 않았지만 위협의 성공 결과가 확인되지 않은 경우에는 `obligation_violation`으로 구분한다. 따라서 검사 의무의 위반, 기록 부족과 성공한 위협을 동일한 결과로 처리하지 않는다. 판정 결과의 전체 의미는 보충 Table S7에 제시한다.

TDID-THR-015는 여러 발급 사건의 상태가 필요한 대표적인 규칙이다. 해당 위협은 동일한 `key proof`가 정책상 허용된 사용 횟수를 넘어 다시 수락되고, 그 결과 별도의 자격증명이 실제로 발급되는 경우를 성공으로 정의한다 [5], [12]. 두 발급 사건 e_1 과 e_2 의 `proof`를 p_1, p_2 라고 하면 설계한 전체 성공 조건은 다음과 같다.

$$\begin{aligned} W_{15}(e_1, e_2) \iff & \text{SameProof}(p_1, p_2) \\ & \wedge \text{ProofValid}(p_2) \\ & \wedge \neg \text{WithinUseLimit}(p_2) \\ & \wedge \text{ExtraIssued}(e_2) \\ & \wedge \text{Issued}(e_2). \end{aligned} \quad (1)$$

`SameProof`는 두 발급 사건의 제한된 `proof` 참조값이 같은 지를 확인하고, `ProofValid`는 두 번째 요청의 `key-proof` 검증 결과를 사용한다. `WithinUseLimit`은 동일 `proof`의 수락 이력과 정책에 정의된 허용 횟수를 비교한다. `ExtraIssued`는 두 번째 요청으로 이전 결과와 구별되는 자격증명이 생성됐는지를 확인하고, `Issued`는 최종 처리 결과가 실제 발급이었음을 요구한다.

다섯 조건이 모두 확인돼야 TDID-THR-015의 `threat_witness`가 성립한다. `replay`된 `proof`가 거부된 경우, 동일 `proof`가 하나의 발급 결과만 역동적으로 반환한 경우 또는 정책이 허용한 범위 안의 사용은 성공 조건에 해당하지 않는다. 반대로 `proof` 비교값, 검증 결과, 사용 이력 또는 발급 결과 가운데 판정에 필요한 정보가 부족하면 해당 실행을 정상으로 간주하지 않고 `incomplete_evidence`로 남긴다.

여러 기록 생산 주체가 필요한 위협도 같은 절차로 판정한다. 예를 들어 TDID-THR-019는 `direct_post` 제시에서 요청이 중계됐는지, 요청을 시작한 세션과 응답을 완료한 세션이 다른지, 세션 연결 검사가 실패했는지, 공격자 세션에서 결과가 사용됐는지와 최종 수락 여부를 함께 확인해야 한다. 해당 값은 검증자와 지갑 등 서로 다른 구성요소에서 생성될 수 있으므로, 허용된 사건 관계를 확인할 수 있을 때만 하나의 성공 조건으로 결합한다. TDID-THR-019는 여러 생산자 기록을 연결하는 설계 예시이며 제6장의 동적 평가 대상에는

포함하지 않는다.

식 (1)은 TDID-THR-015에 대해 설계한 다섯 성공 조건 전체를 나타낸다. 제5장과 제6장의 동적 평가는 이 가운데 `WithinUseLimit`, `ExtraIssued`와 `Issued`의 세 조건만 구현하므로, 해당 결과를 식 (1) 전체에 대한 탐지 결과로 해석하지 않는다.

V. IMPLEMENTATION

본 장은 제4장의 T-DID 설계를 실제 프로토타입과 두 발급자에 적용한 방법을 설명한다. 공통 보안 이벤트와 상태 기반 판정 엔진은 Python으로 구현했으며, 동적 연동은 TDID-THR-015의 `walt.id`와 Credo TS 발급자에 한정한다. 다른 13개 구현체는 제3장의 고정 소스 분석 대상이며 본 장에서 어댑터나 동적 판정 규칙을 구현하지 않았다.

A. Prototype Runtime

T-DID 프로토타입은 Python 3.12로 구현했으며, 입력 검증, 정책 처리, 상태 관리, 규칙 평가와 판정 기록 생성을 독립된 모듈로 구성했다. 공통 보안 이벤트는 `Pydantic` 모델과 `JSON Schema 2020-12`를 모두 통과해야 하며, 입력 검증이 완료되기 전에는 상태를 갱신하거나 규칙을 실행하지 않는다. 등록되지 않은 필드와 검사 이름, 잘못된 자료형 및 허용되지 않은 확장 필드는 입력 오류로 처리한다.

정책 프로파일은 실행에 필요한 검사와 임계값을 정의한다. 어댑터는 정책에 없는 검사를 추가하거나 관측되지 않은 값을 기본값으로 생성하지 않는다. 필요한 값이 없으면 해당 필드는 확보되지 않은 상태로 남기고 판정 단계에서 `incomplete_evidence`의 근거로 사용한다. 입력 자체의 형식, 생산 주체 또는 무결성 검증이 실패한 경우에는 제4장의 Algorithm 2에 따라 `input_rejected`를 반환한다.

상태 저장소는 같은 요청 또는 객체에 속하는 사건의 이전 결과, 사용 횟수와 처리 순서를 유지한다. 동일한 `event_id`가 다시 입력되면 같은 사건으로 상태를 두 번 갱신하지 않는다. 현재 프로토타입은 사건을 한 건씩 처리하고 상태를 프로세스 메모리에 보관한다. 지연된 사건도 유효한 입력이면 상태에 반영하지만 이미 생성된 판정 기록을 소급하여 삭제하거나 변경하지 않는다. 따라서 지연된 사건으로 새로운 판정이 생성되면 기존 판정과 함께 확인해야 한다.

규칙의 대상 위협, 필요한 사건과 상태, 예외 및 출력 종류는 `YAML`로 정의하고, 실제 조건 평가는 Python 함수로 구현했다. 판정 기록에는 적용한 규칙, 조건별 결과, 근거가 된 사건과 확보하지 못한 값을 함께 저장한다. 이 구조는 규칙 정의와 구현체별 입력 변환을 분리하여 동일한 판정 로직이 서로 다른 구현체의 기록을 공통 필드를 통해 읽도록 한다.

B. Adapters and Runtime Information Sources

구현체별 어댑터는 입력 변환을 `lift`와 `enrich` 단계로 나눈다. `lift`는 원본 로그, 함수 반환값과 처리 결과에서

실제로 관측된 값만 읽는다. 이 단계에서는 정책이나 이전 사건을 이용해 값을 보완하지 않는다. *enrich*는 관측된 값에 적용 정책, 구성요소 역할과 구현체 정보를 결합하여 공통 보안 이벤트를 생성한다.

각 필드에는 값의 생성 위치를 구분하기 위한 *provenance*를 기록한다. 구현체가 기본적으로 제공한 값은 *Native*, 시험용 연동 코드가 실행 중 읽어 기록한 값은 *Instrumented*, T-DID가 관측값과 로컬 상태에서 계산한 값은 *Derived*로 표시한다. 다른 구성요소에서 받은 값은 *External*, 정책에서 읽은 값은 *Configured*, 필요한 값을 얻지 못한 경우는 *Unavailable*로 표시한다. 해당 구분은 값의 신뢰도를 자동으로 결정하는 등급이 아니라 판정에 사용한 정보의 생성 경로를 추적하기 위한 것이다. 전체 공통 필드와 등록된 *provenance* 범위는 Supplementary Tables S8–S10에 제시한다.

proof, *credential*과 접근 토큰의 원문은 공통 보안 이벤트에 저장하지 않는다. 동일성 비교가 필요한 경우에는 정해진 운영 범위에서만 비교할 수 있는 참조값을 생성하고 원문은 어댑터 이후 단계로 전달하지 않는다. 사건 연결에 사용하는 로컬 참조값도 외부 판정 결과에 기본적으로 포함하지 않으며, 결과를 공유할 때는 허용된 판정 필드만 선택한다.

C. Integrated Issuer Implementations

실제 T-DID 연동은 TDID-THR-015의 *walt.id identity*와 *Credo TS* 두 발급자를 대상으로 한다. 분석과 실행에 사용한 *revision*은 Table II에 제시한 고정 버전과 동일하다 [31], [36]. 두 구현체에서는 제4장의 공통 이벤트 구조와 판정 절차를 유지하고, 필요한 값을 읽는 연동 위치만 구현체에 맞게 달리했다.

walt.id 연동은 고정된 발급자 서비스를 HTTP 인터페이스로 실행하고 요청과 발급 결과에서 평가에 필요한 값을 수집한다. 시험 하네스는 발급 요청과 응답을 같은 워크플로에 연결하고, *credential* 결과를 원문 대신 비교용 참조값으로 변환한다. T-DID는 발급자 서비스의 기존 발급 결정을 변경하지 않으며 HTTP 요청과 응답이 완료된 뒤 별도의 판정 경로에서 동작한다.

*Credo TS*는 라이브러리 형태로 사용되므로 고정 소스의 별도 작업 사본에 시험용 연동 파일을 두었다. 연동 지점에서는 발급 처리 중 필요한 결과와 종료 상태를 받아 공통 이벤트로 전달한다. 분석 대상으로 고정한 기존 제품 소스의 *proof* 검증이나 발급 결정 로직은 변경하지 않았다. *proof*와 *credential*은 연동 지점에서 참조값으로 변환하고 원문은 T-DID 상태 저장소에 보관하지 않는다.

두 연동은 입력을 확보하는 위치가 서로 다르지만 공통 이벤트의 의미와 상태 기반 판정 로직은 동일하게 유지한다. 따라서 구현체별 차이는 판정 규칙이 아니라 어댑터와 값의 *provenance*에 나타난다.

D. THR-015 Case-Study Implementation

제4장의 식 (1)은 TDID-THR-015의 전체 성공 조건을 *SameProof*, *ProofValid*, *WithinUseLimit*, *ExtraIssued*와 *Issued*의 다섯 항으로 정의한다. 동적 평가용 프로토타입은 이 가운데 마지막 세 조건만 구현했다. Table III은 설계 조건과 실제 평가 범위를 구분한다.

TABLE III
EVALUATED THR-015 CONDITIONS.

Condition	Evaluation value	Evaluated
<i>SameProof</i>	Serialized proof equality not collected	No
<i>ProofValid</i>	Key-proof validation result not collected	No
<i>WithinUseLimit</i>	Distinct issuance count vs. allowed count	Yes
<i>ExtraIssued</i>	Additional distinct credential result	Yes
<i>Issued</i>	Issuer terminal issuance result	Yes

평가기는 *workflow_id*와 동일한 발급 권한 범위를 나타내는 *authorization_commitment*를 이용해 관련 발급 사건을 묶는다. 상태에는 해당 범위에서 확인한 서로 다른 *credential* 결과의 집합, 정책상 허용 횟수와 마지막 갱신 시각을 유지한다. 평가 정책의 허용 횟수는 1로 고정했으며, 동일 결과가 중복 전달되면 새로운 발급으로 세지 않는다. 상태가 재시작됐거나 필요한 이전 사건이 보존 범위 밖에 있어 사용 이력을 확인할 수 없으면 이전 발급을 추정하지 않고 기록 부족으로 처리한다.

*WithinUseLimit*의 평가 구현은 동일한 발급 권한 범위에서 확인한 서로 다른 발급 결과 수와 정책의 허용 횟수를 비교한다. *ExtraIssued*는 첫 번째 결과와 구별되는 추가 *credential* 결과가 존재하는지 확인하고, *Issued*는 해당 요청이 발급자의 종료 결과에서 실제 발급으로 끝났는지를 확인한다. 세 항은 각각 허용 범위 초과, 추가 결과의 존재와 최종 발급 여부를 나타내므로 판정 기록에서 별도로 유지한다.

평가 입력에는 직렬화된 *proof*의 동일성과 *key-proof* 검증 결과를 포함하지 않았다. 따라서 평가 프로그램이 세 조건을 모두 확인하더라도 이를 식 (1)의 전체 *threat_witness*로 해석하지 않는다. 제6장에서는 설계된 다섯 조건 전체와 구분하기 위해 세 평가 조건이 확인된 결과를 *evaluated-condition witness*로 보고한다. 이 범위는 Supplementary Table S6에도 동일하게 정리한다.

VI. EVALUATION

본 장은 T-DID를 다섯 관점에서 평가한다. 먼저 21개 위협과 15개 고정 구현체에서 기본 기록만으로 판정에 필요한 런타임 보안 정보가 어느 범위까지 확보되는지 확인한다. 다음으로 필요한 정보를 추가하고 사건 간 이력을 이용했을 때 위협 조건이 확인된 실행과 정보가 부족한 실행을 구분할 수

있는지 평가한다. 이후 공통 보안 이벤트와 생산자 로컬 정보가 21개 위협의 판정 요구에서 어떻게 도출되는지 분석하고, 제안 로깅과 판정을 적용했을 때의 처리 비용과 구현 재사용 범위를 측정한다.

A. Evaluation Scope and Setup

평가는 서로 다른 세 분석 단위를 구분한다. 첫 번째는 제3장의 21개 위협과 15개 고정 구현체를 조합한 315개 쌍이다. 해당 분석은 위협 성공 여부를 판정하는 데 필요한 기본 런타임 정보의 가용성을 확인하기 위한 것이며, 취약점 수, 제품별 보안 수준 또는 탐지 정확도를 측정하지 않는다.

두 번째는 TDID-THR-015의 walt.id와 Credo TS 발급자를 대상으로 한 primary dynamic evaluation이다. 제4장에서 설계한 다섯 조건 가운데 Table III의 WithinUseLimit, ExtraIssued와 Issued 세 조건을 평가한다. 주 분석은 660개 워크플로로 구성하며, 정상 실행 212개, 허용 범위를 넘는 추가 발급 중 필요한 정보가 완전한 실행 208개와 일부 정보가 누락된 실행 240개를 포함한다. 시간적으로 분리한 holdout은 정상 실행 70개, 정보가 완전한 추가 발급 70개와 정보가 부족한 추가 발급 80개로 구성된 220개 워크플로다. 개발에 사용한 44개 워크플로는 두 분모에서 제외한다.

세 번째는 primary evaluation과 분리한 controlled evaluation이다. 이 실험은 기존 개발 로그에서 판정에 필요한 정보가 부족한 실행에 위협별 런타임 정보를 추가했을 때 판정 가능한 범위가 어떻게 변하는지를 TDID-THR-015와 TDID-THR-019 사례에서 확인한다. 해당 사례는 소규모 통제 실험이며 population-level detector accuracy를 계산하기 위한 표본으로 사용하지 않는다.

B. Native Runtime-Information Availability

Fig. 3는 315개 위협-구현체 조합에서 위협의 적용 여부와 고정 구현체의 기본 자료만으로 전체 판정 정보를 구성할 수 있었는지를 보여준다. *Incomplete native information*은 해당 구현체가 아무런 기록도 생성하지 않는다는 뜻이 아니라, 위협의 성공 조건을 판정하는 데 필요한 정보 가운데 하나 이상이 고정된 기본 자료에서 완성되지 않았음을 의미한다.

315개 조합 가운데 151개에서는 위협 조건이 구현체의 역할 또는 처리 경로에 적용되지 않았다. 나머지 164개 applicable 조합에서는 검사 결과, 객체 관계, 사건 간 이력, 외부 결과 또는 최종 보안 효과 가운데 하나 이상의 정보가 기본 자료만으로 완성되지 않았다. 이는 해당 구현체가 취약하거나 위협을 탐지할 수 없다는 의미가 아니다.

기본 자료와 별도로 추가 계측, 결과 확인 절차와 안전한 실행 환경까지 검토한 결과 22개 조합에서는 통제된 실행 평가를 설계할 수 있었고, 142개에서는 현재 분석 범위에서 필요한 정보 또는 독립적인 결과 확인 절차가 부족했다. 전체 조합의 세부 분석은 Supplementary Table S4b에 제시한다.

이 결과는 위협과 관련된 기능을 구현하고 있다는 사실만으로 실행 단계의 판정이 가능해지는 것은 아님을 보여준다. 실제 판정에는 개별 로그의 존재뿐 아니라 어떤 구성요소가 필요한 값을 생성하는지, 관련 사건을 연결할 수 있는지와 최종 보안 결과를 확인할 수 있는지가 함께 고려되어야 한다.

C. Effect of Added Runtime Information

Primary evaluation에서는 먼저 기본 기록만으로 TDID-THR-015의 세 평가 조건을 함께 확인할 수 있는지 조사했다. 두 발급자의 기본 기록에서는 허용 횟수 초과, 이전 결과와 구별되는 추가 발급 및 최종 발급 결과를 하나의 판정에 필요한 형태로 함께 구성할 수 없었다. 새로운 정보를 추가하지 않고 기본 기록을 공통 형식으로 변환하는 것만으로도 판정 가능한 범위는 증가하지 않았다.

필요한 런타임 정보를 확보한 뒤에는 동일한 추가 정보를 입력받되 이전 발급 사건의 이력을 사용하지 않는 비교 방식과 T-DID를 대조했다. 비교 방식은 현재 사건만 평가하므로 이전 발급과 현재 발급을 연결한 사용 횟수와 추가 발급 여부를 계산하지 않는다. T-DID는 관련 발급 사건의 이력을 유지하여 해당 조건을 평가하고, 판정에 필요한 정보가 부족하면 조건 불성립과 구분하여 *incomplete_evidence*로 기록한다. 두 방식은 사건 간 이력 사용과 정보 부족 처리 방식을 함께 달리하므로, 결과를 두 기능 가운데 하나의 독립적인 효과로 해석하지 않는다.

Table IV은 660개 주 분석 워크플로에서 두 방식이 생성한 결과를 요약한다.

TABLE IV
PRIMARY EVALUATION OUTCOMES.

Workflow group	<i>n</i>	No-history comparison	T-DID
Benign	212	212 not identified	210 no witness; 2 incomplete
Excess issuance, complete information	208	208 not identified	208 evaluated-condition witness
Excess issuance, missing information	240	240 not identified	240 incomplete evidence

허용 범위를 넘는 추가 발급 448개 가운데 208개는 세 평가 조건을 판정하는 데 필요한 정보가 완전했고 240개는 일부 정보가 누락됐다. 사건 이력을 사용하지 않는 비교 방식은 448개 모두에서 세 평가 조건을 확인하지 못했다. T-DID는 정보가 완전한 208개를 *evaluated-condition witness*로 기록하고, 정보가 부족한 240개를 *incomplete_evidence*로 구분했다. 정상 실행 212개에서는 210개가 *no_witness*, 필요한 정보가 부족한 2개가 *incomplete_evidence*로 남았다.

시간 holdout에서도 같은 결과 구분이 나타났다. 정보가 완전한 추가 발급 70개와 정보가 부족한 추가 발급 80개는

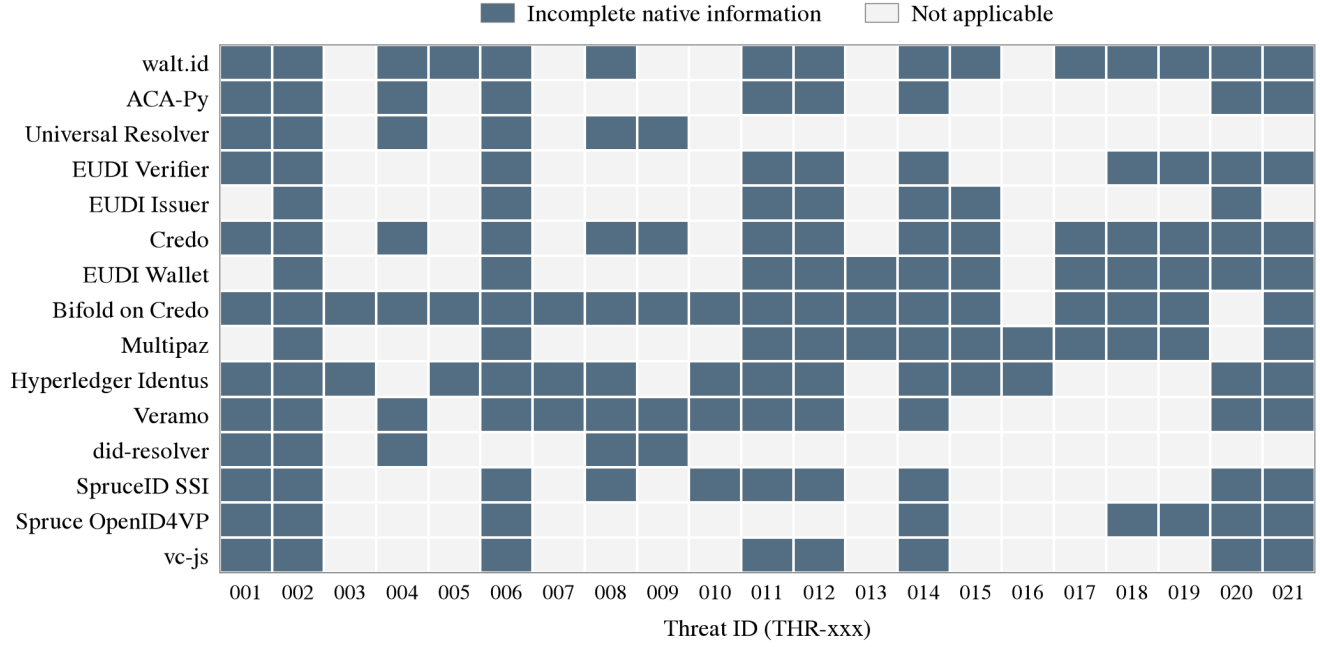


Fig. 3. Native runtime-information availability.

각각 evaluated-condition witness와 incomplete evidence로 구분됐으며, 정상 실행 70개에서는 세 평가 조건이 확인되지 않았다.

Fig. 4은 primary evaluation과 별도로 수행한 controlled evaluation에서 기존 개발 로그와 필요한 런타임 정보를 추가한 조건을 비교한다.

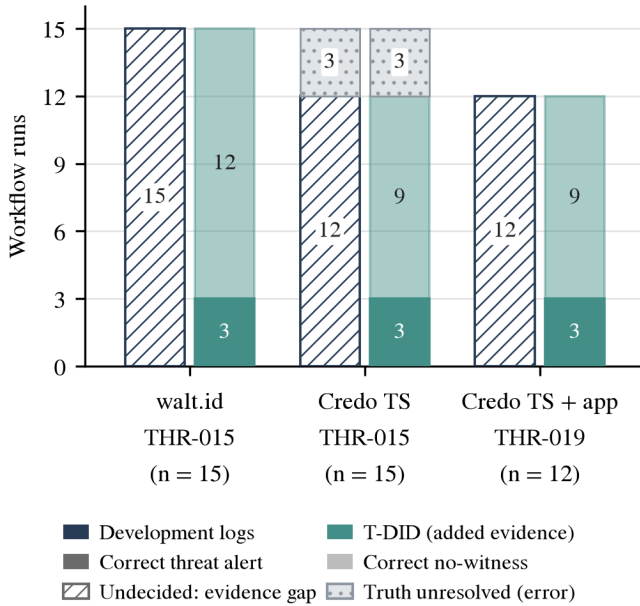


Fig. 4. Effect of added runtime information.

walt.id의 TDID-THR-015 사례에서는 기존 개발 로그만 사용한 15개 실행 모두에서 판정 정보가 부족했지만, 필요한 정

보를 추가한 뒤 3개의 위협 조건 성립 실행과 12개의 조건 불성립 실행을 구분할 수 있었다. Credo TS의 TDID-THR-015에서는 15개 중 3개 실행에서 제품 실행 결과 자체가 해결되지 않았고, 나머지 12개는 추가 정보 적용 후 3개의 위협 조건 성립과 9개의 조건 불성립으로 구분됐다. TDID-THR-019의 12개 사례에서도 기존 로그에서 남은 판정 공백이 추가 정보 적용 후 3개의 위협 조건 성립 실행과 9개의 조건 불성립 실행으로 구분됐다.

Primary evaluation과 controlled evaluation을 함께 보면 공통 형식 자체가 판정 정보를 새로 만들어낸 것은 아니었다. 위협의 성공 조건에서 요구한 런타임 정보를 실제로 확보하고 필요한 사건 관계와 이력을 유지했을 때 판정 가능한 범위가 증가했다.

RQ3의 결과, 두 발급자의 기본 기록은 세 평가 조건에 충분하지 않았으며, 추가 런타임 정보와 사건 간 이력을 이용한 T-DID는 평가 조건이 확인된 실행과 정보가 부족한 실행을 서로 다른 결과로 구분했다.

D. Runtime-Information Model Coverage

Fig. 5은 제4장의 공통 보안 이벤트와 생산자 로컬 정보가 21개 위협의 판정 요구에서 어느 범위까지 사용되는지를 보여준다. 상단은 여러 위협에 반복적으로 나타난 정보 차원을, 하단은 특정 위협군에만 필요한 확장 정보를 나타낸다.

실행 문맥은 21개 위협 모두에서 필요했으며 17개에서는 공통 이벤트 필드로, 4개에서는 생산자 로컬 문맥으로 유지됐다. 보안 검사와 정책은 21개 전체에서 공통 정보로 사용됐다. 최종 결정 또는 위협에서 요구한 보안 효과는 20개 위협에서

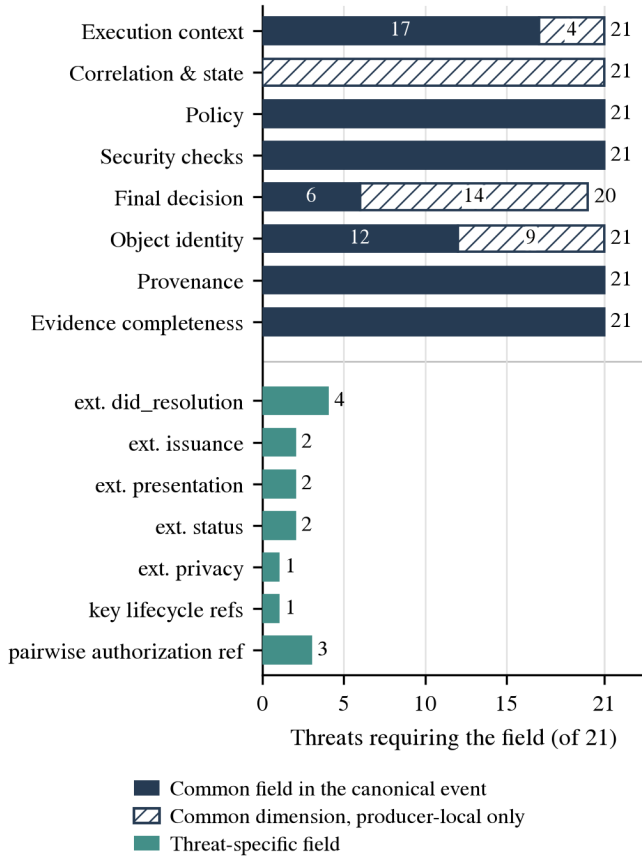


Fig. 5. Runtime-information coverage across threats.

필요했으며, 6개는 공통 최종 결정으로 표현할 수 있고 14개는 해당 효과를 생산자 로컬에서 확인해야 했다. 객체 동일성은 21개 전체에서 요구됐고 12개는 공통 참조값, 9개는 생산자 로컬 비교가 필요했다.

사건 연결과 이력은 21개 위협 모두에서 필요했지만 구체적인 사건 집합, 횡수와 순서는 생산자 로컬에서 유지하도록 했다. 여기서 사건 간 이력이란 공통 이벤트 안의 단일 값이 아니라, 같은 요청이나 객체를 연결하기 위해 생산자가 유지하는 이전 사건의 집합, 횡수 또는 순서를 뜻한다. 모든 위협이 장기간의 전역 이력을 요구하는 것은 아니며, 동일 요청 안의 단계 연결부터 여러 실행에 걸친 누적 이력까지 필요한 범위는 위협마다 다르다.

provenance와 정보 완전성은 21개 전체에서 판정에 사용한 정보의 출처와 결손 여부를 확인하는 공통 요소로 나타났다. 공통 구조만으로 표현하기 어려운 값은 위협군별 확장으로 분리했다. DID 해석 관련 확장은 4개 위협, issuance, presentation과 status 관련 확장은 각각 2개 위협에서 필요했으며, privacy와 key lifecycle 관련 값은 각각 1개 위협에서 사용됐다. pairwise authorization reference는 3개 위협에서 요구됐다.

이 분석은 모든 개별 필드가 최소 필수 집합임을 증명하는 field-ablation 결과가 아니다. 대신 공통 정보 자원과 위협

별 확장이 임의로 선택된 항목이 아니라 21개 위협의 판정 요구에서 추적 가능하게 도출됐으며, 공통 이벤트에 포함할 정보와 생산자 로컬에 유지할 정보가 서로 다름을 보여준다.

E. Processing Cost and Load Response

운영 비용은 동일한 워크플로를 기존 로그만 생성하고 T-DID evaluator를 사용하지 않는 조건과 제안 로깅 및 T-DID 판정을 사용하는 조건에서 실행하여 비교했다. offered load는 초당 1–18 workflow runs로 증가시켰으며, 달성 처리량, p50 및 p95 지연, 측정 구간 종료 시 남은 backlog와 evaluator process RSS를 측정했다. 포화 지점은 달성 처리량이 제공 부하의 95%를 만족하지 못하는 상태가 연속된 두 부하 구간에서 나타날 때 그 첫 지점으로 정했다.

Fig. 6은 두 조건의 부하 응답을 보여준다.

낮은 offered load에서는 두 조건 모두 제공된 부하를 따라가며 backlog 없이 동작했다. 제안 로깅과 T-DID 조건에서는 약 8 workflow runs/s 부근부터 제공 부하와 달성 처리량의 차이, p95 지연과 backlog 증가가 나타났으며, evaluator를 사용하지 않는 조건에서는 약 11 runs/s 부근에서 같은 변화가 나타났다. 포화 이후 제안 조건의 달성 처리량은 약 6–7 runs/s 부근에서 유지됐고, evaluator-off 조건은 약 9 runs/s 수준을 유지했다.

메모리 사용 역시 제안 조건에서 증가했다. evaluator process RSS는 제안 로깅과 사건 이력 기반 판정이 활성화된 뒤 약 190 MiB 수준에서 유지됐고, evaluator-off 조건은 고부하 구간에서 약 65–70 MiB 수준을 보였다. 해당 값은 evaluator 전체 프로세스의 RSS이며 개별 워크플로당 메모리 사용량으로 해석하지 않는다.

제안 구조의 실행 비용이 없는 것은 아니지만, 평가한 낮은 부하 범위에서는 로깅과 판정을 함께 적용한 상태에서도 제공된 워크플로를 backlog 없이 처리했다. 높은 부하에서는 대조 조건보다 이른 포화와 더 높은 메모리 사용이 관찰됐으므로, 현재 단일 프로세스 프로토타입에서는 사건 이력 관리와 기록 처리 경로가 운영 여유를 결정하는 주요 최적화 대상이다.

RQ4의 결과, 두 발급자의 기록 수집 위치는 달랐지만 동일한 공통 정보 구조와 판정 절차를 사용할 수 있었으며, 제안 로깅과 T-DID는 낮은 부하 구간에서 안정적인 처리를 유지하는 한편 높은 부하에서는 측정 가능한 처리량, 지연과 메모리 trade-off를 보였다.

F. Bounded Implementation Reuse

마지막으로 Fig. 7은 평가 경로를 구현하는 데 사용된 non-blank lines of code를 공통 부분과 위협 및 제품별 추가 부분으로 나누어 보여준다. LOC는 구현 품질이나 성능 지표가 아니라 실제로 어느 코드가 공유되고 어느 부분이 새로 필요했는지를 설명하기 위한 공학적 지표로 사용한다.

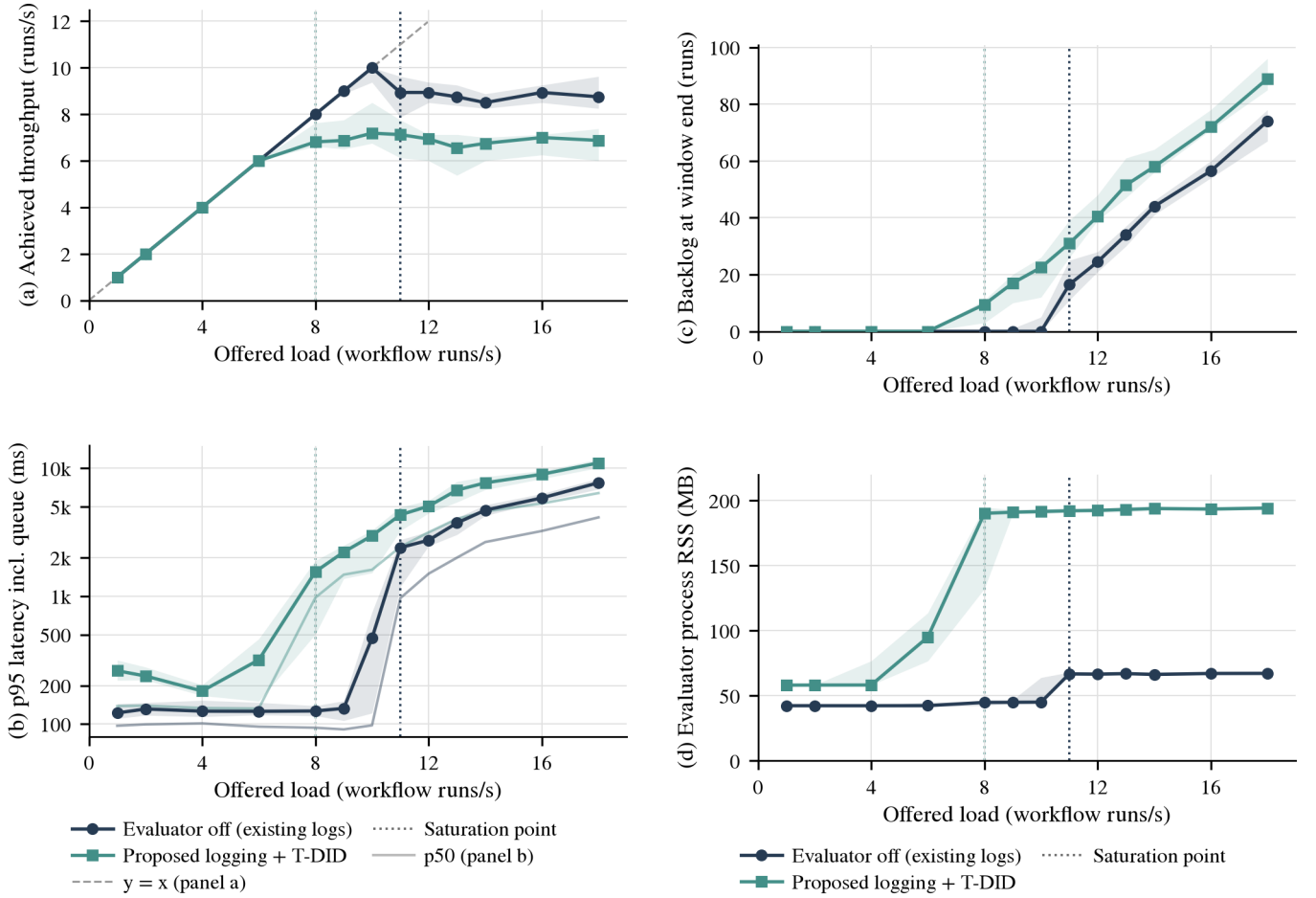


Fig. 6. Load response with and without T-DID.

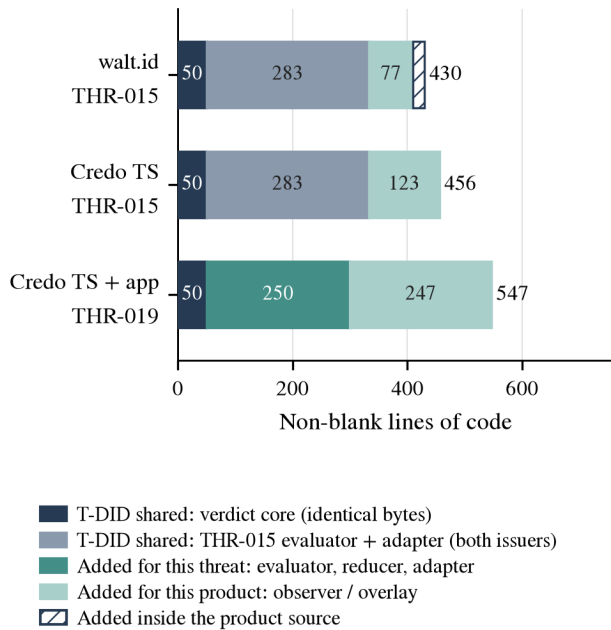


Fig. 7. Implementation reuse across evaluated integrations.

TDID-THR-015의 walt.id와 Credo TS 연동에서는 50 LOC의 verdict core와 283 LOC의 THR-015 evaluator 및 adapter가 동일하게 사용됐다. walt.id에서는 여기에 77 LOC의 제품별 observer/overlay와 20 LOC의 제품 내부 연동 코드가 추가되어 총 430 LOC가 사용됐다. Credo TS에서는 123 LOC의 제품별 observer/overlay가 추가되어 총 456 LOC가 사용됐다. 따라서 실행 구조가 다른 두 발급자에서도 동일 위협의 판정 코드 대부분을 공유할 수 있었다.

TDID-THR-019의 Credo TS와 relying-application 사례에서는 동일한 50 LOC verdict core를 유지하면서, 세션 중계와 여러 구성요소의 관계를 처리하기 위한 250 LOC의 위협별 evaluator, reducer 및 adapter와 247 LOC의 제품 및 application 관측 코드가 추가되어 총 547 LOC가 사용됐다.

이 결과는 T-DID 전체 구현이 새로운 위협과 제품에 수정 없이 재사용된다는 것을 의미하지 않는다. 대신 같은 위협을 서로 다른 발급자에 적용할 때 상당한 판정 코드를 공유할 수 있었고, 다른 정보 topology를 갖는 위협에서도 공통 verdict 처리 위에 위협별 사건 이력 처리와 제품별 관측 지점을 추가하는 방식으로 확장할 수 있음을 보여준다. 이는 구현체별 어댑터와 위협별 판정 로직을 분리한 구조를 발전시키는 것

이 의미 있음을 제한된 범위에서 뒷받침한다.

VII. DISCUSSION AND LIMITATIONS

A. Practical Implications

제6장의 결과는 DID/VC 위협 판정에서 공통 로그 형식보다 먼저 필요한 것이 위협의 성공 조건을 확인할 수 있는 런타임 보안 정보임을 보여준다. 구현체가 관련 기능을 제공하더라도 검사 결과, 최종 보안 결과, 객체 관계 또는 사건 간 이력이 기본 기록에서 함께 확보되지 않으면 특정 실행의 위협 성공 여부를 판단하기 어렵다. 따라서 운영 환경에서는 적용할 위협을 먼저 정하고, 필요한 정보를 어느 구성요소가 생성하며 어떤 범위에서 연결할 수 있는지를 배포 전에 확인할 필요가 있다.

T-DID는 모든 원본 기록을 중앙에 모으는 것을 전제로 하지 않는다. 공통으로 전달할 수 있는 검사 결과, 정책, 판정 결과와 기록 출처는 공통 보안 이벤트로 표현하고, 객체 비교나 반복 사용 이력처럼 범위가 제한되어야 하는 정보는 생산자 로컬에서 유지할 수 있다. 이는 구현체마다 기록 위치가 달라도 판정 규칙이 동일한 보안 의미를 사용하도록 하면서, 불필요한 원문 proof나 credential의 공유를 줄이는데 도움이 된다.

또한 `incomplete_evidence`는 성공 조건이 성립하지 않은 `no_witness`와 구분할 필요가 있다. 필요한 정보가 없는 실행을 정상으로 처리하면 기록 공백이 부정 판정으로 흡수될 수 있기 때문이다. Primary evaluation에서 추가 런타임 정보와 사건 간 이력을 적용한 뒤 완전한 정보와 정보 부족이 서로 다른 결과로 구분된 것은 이러한 판정 의미의 필요성을 보여준다.

운영 비용은 존재하지만 평가한 낮은 부하 범위에서는 보안 로깅과 판정을 적용한 상태에서도 backlog 없이 워크플로를 처리할 수 있었다. 반면 높은 부하에서는 대조 조건보다 이른 포화과 메모리 증가가 나타났다. 따라서 현재 프로토타입은 적용 가능성을 보여주는 동시에, 실제 대규모 배포에서는 사건 이력 관리와 기록 처리 경로의 병렬화 및 비동기화가 필요함을 시사한다. 구현 측면에서도 동일 위협을 두 발급자에 적용할 때 판정 코드의 상당 부분을 공유할 수 있었지만, 다른 위협에는 새로운 사건 관계와 제품별 관측 지점이 필요했다. T-DID의 확장은 공통 구조의 재사용과 위협·구현체별 추가 구현을 함께 고려해야 한다.

B. Limitations and Future Work

본 연구의 21개 위협과 15개 구현체는 DID 문서와 해석, credential status, 발급 및 제시의 서로 다른 처리 경로를 포함하도록 목적 표집했다. 따라서 315개 조합의 분포는 DID/VC 생태계 전체의 취약점 비율이나 제품별 보안 수준을 나타내지 않는다. 소스 분석도 Table II에 제시한 고정 revision과 검토한 처리 경로에 한정되며, 이후 버전, 플러그인, 배포 설정과 외부 의존성에서는 다른 결과가 나타날 수 있다.

Primary dynamic evaluation은 TDID-THR-015의 `walt.id`와 Credo TS 두 발급자에서 설계한 다섯 조건 중 세 조건을 대상으로 수행했다. 따라서 해당 결과를 TDID-THR-015 전체 또는 21개 위협에 대한 탐지 정확도로 일반화할 수 없다. TDID-THR-019와 전체 predicate 검증은 별도의 통제 사례에서만 다뤘으며 population-level precision, recall 또는 false-positive rate를 제공하지 않는다. 또한 사건 간 이력을 사용하지 않는 비교 방식과 T-DID는 이력 사용과 정보 부족 처리 방식을 함께 달리하므로 각 기능의 독립적인 인과효과를 분리하지 않았다.

Fig. 5의 분석은 공통 정보 차원과 위협별 확장이 판정 요구에서 도출됐음을 보여주지만, 모든 개별 필드가 최소 필수 집합임을 증명하지는 않는다. 더 작은 동등 구조가 존재하는지 또는 특정 필드를 다른 정보로 대체할 수 있는지는 별도의 제거 실험이 필요하다.

성능 결과도 현재의 단일 프로세스 연구용 프로토타입에 한정된다. RSS는 프로세스 전체 메모리이며 워크플로 하나의 메모리 사용량이 아니고, Fig. 6의 포화점도 실제 운영 서비스의 최대 처리량을 의미하지 않는다. 마지막으로 T-DID는 식별 가능한 기록 생산 주체가 실제 처리에 부합하는 값을 생성하고 수집 및 판정 경로가 보호된다는 가정을 둔다. 생산자가 의도적으로 거짓 기록을 생성하거나 사건을 숨기는 경우와 장기간의 활동 연결에 따른 개인정보 위협은 별도의 신뢰 및 데이터 관리 정책이 필요하다.

후속 연구에서는 TDID-THR-015의 SameProof와 ProofValid를 primary workflow에 포함하여 다섯 조건 전체를 동일한 평가 경로에서 검증하고, TDID-THR-019처럼 여러 기록 생산 주체가 필요한 위협을 공통 보안 이벤트와 일반 판정 경로에서 end-to-end로 평가할 필요가 있다. 이후 추가 위협과 구현체로 범위를 확장하고, 사건 간 이력과 정보 부족 처리의 효과를 분리한 요인 실험을 수행할 수 있다. 운영 측면에서는 분산 상태 저장소, 비동기 수집과 병렬 판정을 적용해 높은 부하에서의 처리량과 메모리 비용을 줄이는 것이 주요 과제이다.

VIII. CONCLUSION

본 연구는 DID/VC 위협의 분류와 특정 실행에서의 위협 성공 여부 판정이 서로 다른 수준의 정보를 요구한다는 점에 주목하였다. 1차 자료에서 선정한 21개 위협을 ATT&CK, AADAPT, CAPEC와 CWE에 대응시킨 뒤, 15개 고정 버전 구현체와 조합한 315개 쌍을 분석하여 위협별 성공 조건에 필요한 런타임 보안 정보, 기록 생산 주체와 사건 관계를 확인했다. 315개 가운데 151개는 위협 조건이 구현체의 역할 또는 처리 경로에 적용되지 않았고, 나머지 164개에서는 기본 자료만으로 전체 성공 조건에 필요한 정보가 완성되지 않았다. T-DID는 구현체마다 다른 기록을 공통 보안 이벤트로 변환하고, 검사 결과와 최종 결정, 기록 출처 및 필요한 사건 간 이력을 결합하여 위협의 성공 조건을 판정한다. 판정

에 필요한 정보가 부족한 경우에는 이를 정상으로 추정하지 않고 조건 불성립과 구분된 결과로 남긴다.

TDID-THR-015의 walt.id와 Credo TS 두 발급자를 대상으로 한 primary evaluation에서는 설계한 다섯 조건 중 세 조건을 평가했다. 660개 워크플로에서 정보가 완전한 추가 발급 208개는 세 평가 조건이 확인된 실행으로, 정보가 부족한 추가 발급 240개는 incomplete_evidence로 구분됐으며, 정상 실행 212개 중 210개에서는 평가 조건이 성립하지 않았고 2개는 정보 부족으로 남았다. 220개 시간 holdout에서도 같은 구분이 반복됐다. 정보요소 분석에서는 공통 보안 이벤트와 생산자 로컬 정보가 21개 위협의 판정 요구에서 추적 가능하게 도출됨을 확인했으며, 구현체별 관측 위치가 달라 동일한 위협의 판정 로직 상당 부분을 공유할 수 있었다. 부하 평가에서는 낮은 부하 범위에서 제안 로깅과 판정을 적용한 상태로 워크플로를 처리할 수 있었고, 높은 부하에서는 대조 조건보다 이른 포화와 높은 메모리 사용이 나타나 사건 이력 관리와 기록 처리 경로의 최적화 필요성을 확인했다.

결과적으로 DID/VC 런타임 보안 판정에서는 공통 로그 형식 자체보다 어떤 정보가 위협의 성공을 입증하는지, 해당 정보를 어느 구성요소가 생성하는지, 서로 다른 사건을 어떤 범위에서 연결해야 하는지와 필요한 정보가 없을 때 이를 어떻게 처리하는지를 함께 정의하는 것이 중요하다. T-DID는 이러한 정보를 구현체별 실행 기록과 연결하여 판정 가능한 범위를 명시하고, 향후 더 많은 DID/VC 위협과 구현체를 대상으로 런타임 보안 모니터링을 확장하기 위한 기반을 제공한다.

REFERENCES

- [1] World Wide Web Consortium, “Decentralized Identifiers (DIDs) v1.0: Core Architecture, Data Model, and Representations,” World Wide Web Consortium, W3C Recommendation REC-did-core-20220719, Jul. 2022. [Online]. Available: <https://www.w3.org/TR/2022/REC-did-core-20220719/>
- [2] —, “Decentralized Identifier Resolution v0.3,” World Wide Web Consortium, W3C Working Draft WD-did-resolution-20260611, Jun. 2026, working Draft, June 11, 2026. [Online]. Available: <https://www.w3.org/TR/2026/WD-did-resolution-20260611/>
- [3] —, “Verifiable Credentials Data Model v2.0,” World Wide Web Consortium, W3C Recommendation REC-vc-data-model-2.0-20250515, May 2025. [Online]. Available: <https://www.w3.org/TR/2025/REC-vc-data-model-2.0-20250515/>
- [4] —, “Bitstring Status List v1.0,” World Wide Web Consortium, W3C Recommendation REC-vc-bitstring-status-list-20250515, May 2025. [Online]. Available: <https://www.w3.org/TR/2025/REC-vc-bitstring-status-list-20250515/>
- [5] OpenID Foundation, “OpenID for Verifiable Credential Issuance 1.0,” OpenID Foundation, Final Specification, Sep. 2025. [Online]. Available: https://openid.net/specs/openid-4-verifiable-credential-issuance-1_0-final.html
- [6] —, “OpenID for Verifiable Presentations 1.0,” OpenID Foundation, Final Specification, Jul. 2025. [Online]. Available: https://openid.net/specs/openid-4-verifiable-presentations-1_0.html
- [7] MITRE Corporation, “MITRE ATT&CK Enterprise Techniques, Version 19.2,” Data release, 2026, accessed: Aug. 12, 2026. [Online]. Available: <https://attack.mitre.org/resources/attack-data-and-tools/>
- [8] —, “Adversarial Actions in Digital Asset Payment Technologies (AADAPT) Site Data Snapshot,” Data snapshot, build 1761932274, 2026, accessed: Aug. 12, 2026. [Online]. Available: <https://aadapt.mitre.org/>
- [9] —, “Common Attack Pattern Enumeration and Classification, Version 3.9,” Data release, 2026, accessed: Aug. 12, 2026. [Online]. Available: <https://capec.mitre.org/data/archive.html>
- [10] —, “Common Weakness Enumeration, Version 4.20,” Data release, 2026, accessed: Aug. 12, 2026. [Online]. Available: <https://cwe.mitre.org/data/downloads.html>
- [11] W3C Security Interest Group, “Threat Model for Decentralized Credentials,” World Wide Web Consortium, W3C Group Note Draft DNOTE-threat-model-decentralized-credentials-20260622, Jun. 2026, group Note Draft, June 22, 2026. [Online]. Available: <https://www.w3.org/TR/2026/DNOTE-threat-model-decentralized-credentials-20260622/>
- [12] T-DID Project, “Detector-Fit Threat Baseline: Stable Identifier Consolidation and Boundary Audit,” Supplementary research artifact, 2026, supplementary research artifact; public artifact URL to be added before submission.
- [13] —, “Five-Element ATT&CK and DID/VC Taxonomy Reconciliation,” Supplementary research artifact, 2026, supplementary research artifact; public artifact URL to be added before submission.
- [14] OpenWallet Foundation, “Result of Presentation Verification Not Checked for W3C Format JSON-LD LDP-VC,” GitHub Security Advisory, Jan. 2024, gHSA-97x9-59rv-q5pm; associated with CVE-2024-21669; accessed Aug. 12, 2026. [Online]. Available: <https://github.com/openwallet-foundation/acapy/security/advisories/GHSA-97x9-59rv-q5pm>
- [15] CVE Program, “CVE-2024-21669,” CVE Record, 2024, published Jan. 11, 2024; accessed Aug. 12, 2026. [Online]. Available: <https://www.cve.org/CVERecord?id=CVE-2024-21669>
- [16] OpenTelemetry Authors, “OpenTelemetry Logs Data Model,” Cloud Native Computing Foundation, Specification 1.50.0, Oct. 2025, fixed commit b4bb23c6f2a5e30d3b729cfc425531d4262a6afa. [Online]. Available: <https://github.com/open-telemetry/opentelemetry-specification/blob/v1.50.0/specification/logs/data-model.md>
- [17] Open Cybersecurity Schema Framework, “Open Cybersecurity Schema Framework,” Open Cybersecurity Schema Framework, Schema specification 1.6.0, Aug. 2025, fixed commit d0cd8a0fef198bf93086044e33fda6d80c74b9ef. [Online]. Available: <https://github.com/ocsf/ocsf-schema/tree/1.6.0>
- [18] Elastic, “Elastic Common Schema,” Elastic, Schema specification 9.1.0, Aug. 2025, fixed commit d3d3c52fe57c329f17ea9c619724e55f460d8f6d. [Online]. Available: <https://github.com/elastic/ecs/tree/v9.1.0>
- [19] Cloud Native Computing Foundation, “CloudEvents – Version 1.0.2,” Cloud Native Computing Foundation, Specification 1.0.2, Aug. 2025, fixed commit fc1f6f31f5f011a72183f1bcea20c987cb683ade. [Online]. Available: <https://github.com/cloudevents/spec/blob/v1.0.2/cloudevents/spec.md>
- [20] D. Hardt, “The OAuth 2.0 Authorization Framework,” RFC Editor, RFC 6749, Oct. 2012. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6749>
- [21] E. Bartocci, J. Deshmukh, A. Donzé, G. Fainekos, O. Maler, D. Ničković, and S. Sankaranarayanan, “Specification-Based Monitoring of Cyber-Physical Systems: A Survey on Theory, Tools and Applications,” in *Lectures on Runtime Verification*, ser. Lecture Notes in Computer Science. Cham: Springer International Publishing, 2018, vol. 10457, pp. 135–175. [Online]. Available: https://doi.org/10.1007/978-3-319-75632-5_5
- [22] M. Echeverria, Z. Ahmed, B. Wang, M. F. Arif, S. R. Hussain, and O. Chowdhury, “PHOENIX: Device-centric cellular network protocol monitoring using runtime verification,” arXiv:2101.00328, 2021. [Online]. Available: <https://arxiv.org/abs/2101.00328>

- [23] Q. Lan, A. Kaul, S. Jones, and S. Westrum, “Zero-trust runtime verification for agentic payment protocols: Mitigating replay and context-binding failures in AP2,” arXiv:2602.06345, 2026. [Online]. Available: <https://arxiv.org/abs/2602.06345>
- [24] M. Farrell, M. Bradbury, R. C. Cardoso, M. Fisher, L. A. Dennis, C. Dixon, A. T. Sheik, H. Yuan, and C. Maple, “Security-Minded Verification of Cooperative Awareness Messages,” *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 4, pp. 4048–4065, Jul. 2024. [Online]. Available: <https://doi.org/10.1109/TDSC.2023.3345543>
- [25] U. Zdun, P.-J. Queval, G. Simhandl, R. Scandariato, S. Chakravarty, M. Jelić, and A. Jovanović, “Detection Strategies for Microservice Security Tactics,” *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 3, pp. 1257–1273, May 2024. [Online]. Available: <https://doi.org/10.1109/TDSC.2023.3276487>
- [26] S. Huh, M. Shim, J. Lee, S. S. Woo, H. Kim, and H. Lee, “DID We Miss Anything?: Towards privacy-preserving decentralized ID architecture,” *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 6, pp. 4881–4898, 2023.
- [27] OASIS Cyber Threat Intelligence Technical Committee, “STIX Version 2.1,” OASIS, OASIS Standard, Jun. 2021. [Online]. Available: <https://docs.oasis-open.org/cti/stix/v2.1/os/stix-v2.1-os.html>
- [28] World Wide Web Consortium, “PROV-DM: The PROV Data Model,” World Wide Web Consortium, W3C Recommendation REC-prov-dm-20130430, Apr. 2013. [Online]. Available: <https://www.w3.org/TR/2013/REC-prov-dm-20130430/>
- [29] —, “Trace Context,” World Wide Web Consortium, W3C Recommendation REC-trace-context-1-20211123, Nov. 2021. [Online]. Available: <https://www.w3.org/TR/2021/REC-trace-context-1-20211123/>
- [30] T-DID Project, “Fixed-Commit DID/VC Implementation Sampling Frame,” Supplementary research artifact, 2026, supplementary research artifact; public artifact URL to be added before submission.
- [31] walt.id, “waltid-identity Source Snapshot,” Git repository, commit f5b5995059c4ab69d6bc73f434c72d1574188963, 2026, accessed: Aug. 12, 2026. [Online]. Available: <https://github.com/walt-id/waltid-identity/tree/f5b5995059c4ab69d6bc73f434c72d1574188963>
- [32] OpenWallet Foundation, “Aries Cloud Agent Python Source Snapshot,” Git repository, commit b5bf8388f80013f2ce3dfdda2a24e200ebd9b717, 2026, accessed: Aug. 12, 2026. [Online]. Available: <https://github.com/openwallet-foundation/acapy/tree/b5bf8388f80013f2ce3dfdda2a24e200ebd9b717>
- [33] Decentralized Identity Foundation, “Universal Resolver Source Snapshot,” Git repository, commit 844752ee6decc3f23d0aa5e52183d576a19bda69, 2026, accessed: Aug. 12, 2026. [Online]. Available: <https://github.com/decentralized-identity/universal-resolver/tree/844752ee6decc3f23d0aa5e52183d576a19bda69>
- [34] European Commission, “EUDI Verifier Endpoint v0.11.0 Source Snapshot,” Git repository release v0.11.0, commit 2bc0bd98a81742db321e96d31d88142128006422, 2026, accessed: Aug. 12, 2026. [Online]. Available: <https://github.com/eu-digital-identity-wallet/eudi-srv-verifier-endpoint/releases/tag/v0.11.0>
- [35] —, “EUDI PID Issuer Source Snapshot,” Git repository, commit c14af675b713acf6a4933fc3d4e169267a77b17c, 2026, accessed: Aug. 12, 2026. [Online]. Available: <https://github.com/eu-digital-identity-wallet/eudi-srv-pid-issuer/tree/c14af675b713acf6a4933fc3d4e169267a77b17c>
- [36] OpenWallet Foundation, “Credo TypeScript Source Snapshot,” Git repository, commit 3c32928107639bc355f82d4ba11c4d91901e11cb, 2026, accessed: Aug. 12, 2026. [Online]. Available: <https://github.com/openwallet-foundation/credo-ts/tree/3c32928107639bc355f82d4ba11c4d91901e11cb>
- [37] European Commission, “EUDI Android Wallet Core Source Snapshot,” Git repository, commit c67b0ebebdc36076b298c372a56d5cc1962658f8, 2026, accessed: Aug. 12, 2026. [Online]. Available: <https://github.com/eu-digital-identity-wallet/eudi-lib-android-wallet-core/tree/c67b0ebebdc36076b298c372a56d5cc1962658f8>
- [38] OpenWallet Foundation, “Bifold Wallet Source Snapshot,” Git repository, commit 41c2c54f8bee313e2cc2bfc7d0a35047b2f562e7, 2026, accessed: Aug. 12, 2026. [Online]. Available: <https://github.com/openwallet-foundation/bifold-wallet/tree/41c2c54f8bee313e2cc2bfc7d0a35047b2f562e7>
- [39] —, “Multipaz Wallet Source Snapshot,” Git repository, commit a0d63e64ed878e1b7a67255617487888e812cf1a, 2026, accessed: Aug. 12, 2026. [Online]. Available: <https://github.com/openwallet-foundation/multipaz-wallet/tree/a0d63e64ed878e1b7a67255617487888e812cf1a>
- [40] Hyperledger Indentus, “Indentus Cloud Agent Source Snapshot,” Git repository, commit 79ca64e6f0ff42f567d97ed5ee3e3d00efdddbdf9, 2026, accessed: Aug. 12, 2026. [Online]. Available: <https://github.com/hyperledger-indentus/cloud-agent/tree/79ca64e6f0ff42f567d97ed5ee3e3d00efdddbdf9>
- [41] Decentralized Identity Foundation, “Veramo Source Snapshot,” Git repository, commit 30089045f3fe26c0b00c7f4512e27729941ea643, 2026, accessed: Aug. 12, 2026. [Online]. Available: <https://github.com/decentralized-identity/veramo/tree/30089045f3fe26c0b00c7f4512e27729941ea643>
- [42] —, “JavaScript DID Resolver Source Snapshot,” Git repository, commit a36ebdb00e5aa5ed7dab53860cfe1c0a79960209, 2026, accessed: Aug. 12, 2026. [Online]. Available: <https://github.com/decentralized-identity/did-resolver/tree/a36ebdb00e5aa5ed7dab53860cfe1c0a79960209>
- [43] SpruceID, “SpruceID SSI Source Snapshot,” Git repository, commit 89630368438c81b55335362b21621d3aadd48d93, 2026, accessed: Aug. 12, 2026. [Online]. Available: <https://github.com/spruceid/ssi/tree/89630368438c81b55335362b21621d3aadd48d93>
- [44] —, “SpruceID OpenID4VP Source Snapshot,” Git repository, commit e5f29b85f14ca0b3c6eb6852e4d2d158f601fd63, 2026, accessed: Aug. 12, 2026. [Online]. Available: <https://github.com/spruceid/openid4vp/tree/e5f29b85f14ca0b3c6eb6852e4d2d158f601fd63>
- [45] Digital Bazaar, “vc-js Source Snapshot,” Git repository, commit 7ddb985a67e42afeb24ef833a5b45ecb72d7771d, 2026, accessed: Aug. 12, 2026. [Online]. Available: <https://github.com/digitalbazaar/vc/tree/7ddb985a67e42afeb24ef833a5b45ecb72d7771d>
- [46] T-DID Project, “Threat-by-Implementation Evidence-Producer Mapping,” Supplementary research artifact, 2026, supplementary research artifact; public artifact URL to be added before submission.
- [47] K. Kent and M. Souppaya, “Guide to Computer Security Log Management,” National Institute of Standards and Technology, NIST Special Publication 800-92, Sep. 2006. [Online]. Available: <https://doi.org/10.6028/NIST.SP.800-92>
- [48] R. Gerhards, “The Syslog Protocol,” RFC Editor, RFC 5424, Mar. 2009. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc5424>
- [49] B. Schneier and J. Kelsey, “Secure Audit Logs to Support Computer Forensics,” *ACM Transactions on Information and System Security*, vol. 2, no. 2, pp. 159–176, May 1999. [Online]. Available: <https://doi.org/10.1145/317087.317089>

IX. BIOGRAPHY SECTION